

VIETNAMESE – GERMAN UNIVERSITY

**Faculty of Engineering
Computer Science Program**

**Design and Pilot Protocol of a Next.js Boilerplate
Framework for AI-Assisted SaaS Development**

Full name: Do Hoang Trieu

Student ID: 10422120

First supervisor: Dr. Le Lam Son

Second supervisor: Dr. Tran Huu Tam

BACHELOR THESIS

Submitted in partial fulfillment of the requirements for the degree of Bachelor Engineering
in study program Computer Science, Vietnamese - German University, 2026

Ho Chi Minh City, Viet Nam

Declaration

I, Do Hoang Trieu, hereby declare that this thesis, entitled “Design and Pilot Protocol of a Next.js Boilerplate Framework for AI-Assisted SaaS Development,” is my own original work. All sources of information have been properly acknowledged and referenced.

In preparing this thesis, I used AI tools to assist with code generation, prose drafting, and information research. I reviewed the output against the original sources and the SmartKit codebase, and take full responsibility for the content.

This thesis has not been submitted, in whole or in part, for the award of any other academic degree at this or any other institution. The author agrees that this thesis, upon acceptance, will be archived in the library of Vietnamese–German University and made available for reference use in accordance with the university’s regulations.

Acknowledgements

Many people supported this thesis. I owe them all a debt I cannot fully repay here.

My supervisor, Dr. Le Lam Son, guided every stage of this work. His pushback on early drafts of the evaluation methodology forced me to rethink what I was actually claiming, and his insistence on reading the rubric before I wrote the pilot saved me from several embarrassing retrofitting mistakes.

The Faculty of Engineering and the Computer Science Program at Vietnamese–German University provided the academic environment and resources that made the work possible. The maintainers of Next.js, Drizzle ORM, and Biome wrote codebases clear enough that I could learn from them by reading, not just by running them. SmartKit exists because their documentation held up under my questions.

My family put up with months of me talking about rubrics and Wilson intervals at dinner. Their patience carried me through the stretches where the thesis felt like it was going nowhere. My classmates and friends at VGU argued with me about React Server Components and AI-assisted development in the corridors of the university; many of the design decisions in the following chapters trace back to those arguments.

I am grateful to the open-source and research community whose published work on AI coding assistants, code generation benchmarks, and framework adoption provided the empirical ground I built on. Anything that is wrong in the pages that follow is mine.

Abstract

This thesis proposes an evaluation framework for AI-generated Next.js boilerplate and ships SmartKit, the boilerplate the framework measures. The pilot ran at N=5 per condition (Condition A: SmartKit; Condition B: convention-naive scaffold) on one feature; it shows the framework produces scores, but the sample is too small to support inferential claims. Larger-N runs, cross-tool replication, and developer-participant studies are future work.

AI coding assistants such as GitHub Copilot, Claude Code, and Cursor are now in daily use. Peng et al. report a 55.8% completion-time gain on a controlled HTTP-server task (**peng2023impact**), and Yetistiren et al. found boilerplate generation to be the strongest AI use case among practitioners (**yetistiren2022assessing**). The benchmarks that dominate the literature—HumanEval, MBPP+, APPS, BigCodeBench, LiveCodeBench for algorithms; SWE-bench for bug-fixing—do not measure whether AI-generated code respects framework conventions. Server/Client boundary violations in Next.js App Router, for instance, fail at runtime in a way no static type check can catch.

This thesis proposes an evaluation framework for AI-generated Next.js boilerplate. The framework scores each generated artefact on three dimensions: **Structural Integrity** (file location, Server/Client boundary, naming), **Type & Contract Safety** (TypeScript error count plus Zod-validation coverage), and **Coverage & Completeness** (presence of required files per feature). Each dimension is measured automatically: Biome for structural rules, `tsc -noEmit` with custom rules for type safety, and a checklist-driven file enumeration for coverage. Each dimension reports a 0–5 score per run.

The framework is instantiated through SmartKit, an open-source Next.js 16.2.12 boilerplate built on Drizzle ORM, Tailwind CSS, Biome, Zod, Resend, and Better Auth. SmartKit encodes feature-based modularity with explicit Server/Client component boundaries and acts as both the framework’s evaluation target and a working SaaS starting point. The thesis contributes a framework proposal, a published artefact (SmartKit), a pilot protocol at N=5 runs per condition (Condition A: SmartKit; Condition B: convention-naive scaffold), and the replication artefacts that operationalise the protocol: a pre-registration document, a baseline-bootstrap script, a scoring rulebook, and the prompt templates used in the pilot. The pilot is a worked-example; the sample size is too small for inferential statistics. Future work includes cross-tool replication, larger-N validation, and developer-participant studies. All scripts and the codebase are released publicly for replication.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
List of Abbreviations	viii
1 Introduction and Motivation	1
1.1 Overview	1
1.2 The Problem: AI Convention Violations in Next.js	1
1.3 Research Objectives and Questions	2
1.4 Scope and Limitations	6
2 Background	8
2.1 AI Coding Assistants	8
2.2 Next.js Framework Architecture	8
2.3 SaaS Boilerplate Patterns	9
3 Related Work	10
3.1 Code Generation Benchmarks	10
3.2 Empirical Studies on Developer Productivity	11
3.3 Evaluation Methodology	12
3.4 Research Gap Summary	12
3.5 Literature Search Methodology	13
3.6 Alternative Approaches to AI-Codebase Alignment	13
4 Proposed Solution	15
4.1 Design Rationale	15
4.2 Feature-Based Modular Architecture	15
4.3 Server/Client Boundary Conventions	16
4.4 Database Schema Patterns	16
4.5 Type-Safe API Contracts	17
5 Implementation	18
5.1 Technology Stack	18
5.2 Project Structure	18
5.3 Key Implementation Patterns	19
6 Evaluation	20
6.1 Evaluation Framework	20
6.2 Design Rationale and Trade-offs	21
6.3 Evaluation Scenarios	22
6.4 Control Comparison	23
6.4.1 Design Rationale	23

6.4.2	Experiment Protocol	24
6.4.3	Expected Threats and Mitigations	25
6.5	Multi-Run Evaluation	25
6.5.1	Non-Determinism Threat	25
6.5.2	Experimental Protocol	26
6.5.3	Statistical Reporting	26
6.5.4	Implications for Framework Reliability	27
6.6	Evaluation Metrics	27
6.7	Pilot Evaluation Protocol	27
7	Conclusion and Future Work	31
7.1	Summary	31
7.2	Limitations and Threats to Validity	31
7.3	Diffusion Pathway and Adoption Risks	33
7.4	Future Work	34
A	Pilot Pre-Registration — SmartKit C2	36
A.1	Pilot Summary	36
A.2	Pilot Feature	36
A.3	Conditions	37
A.3.1	Condition A (SmartKit)	37
A.3.2	Condition B (baseline)	37
A.3.3	Run Schedule	38
A.3.4	Adaptive N=3 Verdict Rule	38
A.3.5	Inter-Rater Reliability	38
A.4	Prompt Template	38
A.5	Scoring Rubric	39
A.5.1	Dimension 1: Structural Integrity (0–5)	40
A.5.2	Dimension 2: Type & Contract Safety (0–5)	40
A.5.3	Dimension 3: Coverage & Completeness (0–5)	41
A.5.4	Aggregate Score	41
A.6	Exclusion Criteria	42
A.7	Lock and Amendment Procedure	42
A.8	Snapshot Limitation	43
A.9	References	43
A.10	Pilot Run Log	44
A.11	Amendment Log	44
A.12	JSON Output Schema	46

List of Figures

1.1 AI-generated Next.js App Router code violates Server/Client boundaries by passing event handlers across component types, producing runtime errors that static analysis cannot detect. 5

1.2 The three evaluation dimensions are orthogonal: structural violations, type errors, and coverage gaps are independently measurable aspects of AI code quality. 6

List of Tables

3.1	Comparison of SmartKit against established Next.js starter templates. . . .	14
6.1	Reference coverage checklist for the <code>isEnabled</code> feature flag utility.	22
6.2	Condition A (SmartKit at full setup) vs. Condition B (convention-naive scaffold) used in the pilot evaluation. Audit anchors: SmartKit cells <code>[a]</code> reference the pre-registration commit hash (<code>pre-reg commit: 0ac4b35</code>); create-next-app cells <code>[b]</code> reference the CLI invocation recorded in <code>setup-condition-b.ts</code> (CLI: <code>pnpm tsx scripts/setup-condition-b.ts</code>).	23
6.3	Pilot scoring output (actual data from completed runs, plus 3 pending Cond A runs per §A.11 amendment 2026-09-06). $N=3$ for Condition B (Runs 1–3); $N=2$ completed for Condition A (Runs 4–5); $N=3$ <i>pending</i> for Condition A (Runs 6–8) per adaptive N and the §A.5.4.1 verdict rule ($N = 5$ per condition).	28
A.1	Pilot summary	36
A.2	Pilot run schedule ($N=5$ per condition, alternating order)	38
A.3	Structural Integrity scoring rubric (0–5)	40
A.4	Type Safety sub-score rubric (0–5)	40
A.5	Contract Safety sub-score rubric (0–5)	41
A.6	Coverage & Completeness scoring rubric (0–5)	41
A.7	Exclusion criteria for pilot runs	42
A.8	Pilot run log. Columns: Str = Structural Integrity, Type = Type & Contract Safety, Cov = Coverage & Completeness, Tot = sum of three dimensions (0–15), DP1/DP2/DP3 = directional prediction verdicts (§A.5.4.1: <i>5/5 confirmed</i> , <i>0/5 refuted</i> , <i>1–4/5 inconclusive</i>), ttc = time-to-correct (minutes from session open to committed snapshot). All runs reached <code>score-flag.ts</code> completion with no exclusion event (E1–E5). Wilson 95% CIs on per-condition pass rates are reported in §6.7 (body).	44
A.9	Amendment log (locked + 7 amendments). Source of truth: <code>docs/pilot-pre-registration.m</code> §11.	45
A.10	JSON output schema for pilot runs (matches SmartKit/ <code>scripts/score-flag.ts</code> v1.2 output). Snake-case field names; per-dimension scores are 0–5.	46

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command-Line Interface
CSS	Cascading Style Sheets
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
LLM	Large Language Model
MBPP	Mostly Basic Python Problems (benchmark)
MBPP+	Extended MBPP benchmark with additional rigorous test cases
ORM	Object-Relational Mapper
RAG	Retrieval-Augmented Generation
SaaS	Software as a Service
SQL	Structured Query Language
SWE-bench	Software Engineering Benchmark (GitHub-issue resolution benchmark)
UI	User Interface
VGU	Vietnamese–German University

1. Introduction and Motivation

The chapter opens with the recent wave of AI coding assistants and the parallel shift in Next.js toward the App Router, then identifies the absence of an evaluation framework for framework-specific convention adherence. From that gap the chapter derives the research objectives, questions, and hypotheses that structure the rest of the thesis, and closes with the scope within which the SmartKit framework is designed and evaluated.

1.1 Overview

Controlled studies report substantial productivity gains from AI coding assistants. Peng et al. showed that developers using GitHub Copilot completed tasks 55.8% faster than control groups (**peng2023impact**), while Yetistiren et al. found AI tools particularly effective for boilerplate-generation tasks (**yetistiren2022assessing**). Next.js has emerged as the dominant React meta-framework for production web applications, with its App Router architecture introduced in version 13 fundamentally changing application structure through React Server Components and explicit Server/Client boundaries (**nextjsdocs**). The failure mode that motivates this thesis is concrete: when an AI coding assistant generates a Server Component that imports a Client Component and passes an event handler as a prop, Next.js produces the runtime error “Event handlers cannot be passed to Client Component props”—a failure that `tsc -noEmit` cannot catch because it has no type-level representation. Practitioners have observed recurring AI generation errors in Next.js App Router code, including authentication security flaws, component boundary violations, and outdated API usage from older training data. Without systematic evaluation, developers cannot assess whether AI-generated Next.js code adheres to framework conventions or will fail in production.

Existing benchmarks address algorithmic problem-solving and bug-fixing capabilities but not framework convention adherence. HumanEval evaluates Python code generation for standalone functions (**chen2021evaluating**); SWE-bench assesses AI ability to resolve GitHub issues (**jimenez2024swebench**); RigorBench measures process-discipline metrics (**rigorbench2026**). None of them capture the specific architectural requirements of Next.js App Router, such as Server/Client component boundaries, type-safe contracts, or file-system routing conventions. The gap means developers have no quantitative basis for comparing AI tools on framework-specific tasks.

Intended users. SmartKit targets solo developers and small teams (2–5 members) working on Next.js projects who use AI coding assistants (primarily Cursor AI and GitHub Copilot) for boilerplate generation. The intended users are intermediate developers who understand React and TypeScript fundamentals but seek to reduce boilerplate setup time and enforce consistent conventions in AI-assisted development. The framework supports incremental adoption: teams can apply the feature-based folder structure and Server/Client boundary conventions without committing to the full stack.

1.2 The Problem: AI Convention Violations in Next.js

This thesis addresses a concrete problem: AI coding assistants consistently generate Next.js App Router code that violates framework conventions, producing runtime errors and

technical debt, yet no evaluation framework exists to measure this adherence. Designing such an evaluation framework faces three principal challenges. First, framework conventions are implicit and distributed across documentation, error messages, and community patterns, making them difficult to enumerate into testable metrics. Second, AI-generated code failures manifest at runtime rather than compile time. Server/Client boundary violations produce “Event handlers cannot be passed to Client Component props” errors only when executed, so they require dynamic analysis rather than static linting alone. Third, framework conventions evolve across versions; Next.js 16 removed synchronous APIs that AI tools continue to generate from older training data, demanding version-aware evaluation criteria.

The focus on Next.js App Router is deliberate. First, market dominance makes it the primary target: community surveys consistently place Next.js among the most widely adopted React meta-frameworks (**nextjsdocs**), and Stack Overflow’s annual Developer Survey has tracked Next.js as the most-used React meta-framework since 2021. Second, training-data mismatch creates measurable failures: AI tools generate outdated patterns from older training data that conflict with current Next.js conventions, particularly around Server/Client boundaries and async API changes. Third, practical feasibility applies: the author has existing Next.js expertise, enabling direct testing of enforcement mechanisms. The pilot targets Cursor AI as the primary case study because its AGENTS.md feature implements documentation-based guidance, which prior research on grounded prompting suggests can improve AI output quality (**barke2022grounded**; **zhang2023practices**).

1.3 Research Objectives and Questions

While Next.js dominates React meta-framework adoption and AI coding assistants such as GitHub Copilot and Cursor AI have demonstrated substantial productivity gains, there is a distinct gap at the intersection of these two areas: a systematic, operationalizable evaluation framework that measures how closely AI-generated Next.js boilerplate code adheres to framework-specific conventions. Existing benchmarks—HumanEval, SWE-bench, RigorBench—address algorithmic problem-solving and bug-fixing capability, but none capture the architectural requirements of Next.js App Router such as Server/Client component boundaries, type-safe contracts, or file-system routing conventions (**chen2021evaluating**; **jimenez2024swebench**; **rigorbench2026**). This thesis targets precisely that gap.

The primary motivation of this thesis is to bridge this gap by designing a three-dimensions evaluation framework that turns an implicit set of framework conventions into measurable, comparable scores, and by instantiating that framework in SmartKit, a structured Next.js boilerplate that provides the necessary enforcement mechanisms and the evaluation target codebase.

To achieve this, the research outlines the following core objectives:

- **Design a three-dimensions evaluation framework.** Construct a framework that scores AI-generated Next.js boilerplate code along three dimensions: Structural Integrity, Type & Contract Safety, and Coverage & Completeness, so that convention adherence becomes a quantitative, comparable property of generated code.
- **Implement SmartKit as a structured architecture.** The thesis releases SmartKit, an open-source Next.js boilerplate that demonstrates feature-based modularity, explicit Server/Client boundaries, and type-safe contracts, providing both the enforcement mechanisms and the target codebase for the framework.

- **Conduct a pilot empirical case study.** The thesis applies the framework to a representative feature generated by Cursor AI under the SmartKit condition to demonstrate that the framework can produce illustrative, actionable scores on real generated code.

Consequently, this research seeks to answer the following research questions:

1. How can an evaluation framework be consistently designed to measure AI coding assistant adherence to Next.js App Router boilerplate conventions along the Structural Integrity, Type & Contract Safety, and Coverage & Completeness dimensions?
2. Can the resulting framework be operationalized into measurable metrics, and does it produce illustrative, actionable scores when applied to representative AI-generated Next.js features?

These objectives and questions give rise to two classes of claims, both documented as design-level pre-registrations to be exercised by the pilot (Section 6.7). The first class states **framework properties**—descriptive claims about whether the framework can measure the dimensions it targets. The second class states **directional predictions**—falsifiable claims about the sign of the score difference between conditions, to be reported as confirmed, refuted, or inconclusive per Section 6.7.

Framework properties. The pilot is designed to demonstrate that the framework can be operationalised on a real artifact:

- **FP1 (Structural Integrity).** The framework can measure file placement, Server/Client boundary usage, and naming patterns in AI-generated Next.js code, producing a 0–5 score per run on a representative feature.
- **FP2 (Type & Contract Safety).** The framework can measure TypeScript error counts and Zod-validation coverage in AI-generated Next.js code, producing a 0–5 score per run that combines a Type-Safety sub-score and a Contract-Safety sub-score via the weakest-link rule.
- **FP3 (Coverage & Completeness).** The framework can enumerate AI-generated files against a feature-specific checklist and produce a 0–5 per-run score that reflects required-artifact presence.

Directional predictions. Each directional prediction states the expected sign of the difference between Condition A (SmartKit at full setup) and Condition B (convention-naive scaffold) on a representative feature—the `isEnabled(flagKey, userId?)` utility, which uses no service that SmartKit integrates. Predictions are evaluated as *confirmed* (all runs in the predicted direction), *refuted* (no runs in the predicted direction), or *inconclusive* (intermediate), per the adaptive-N protocol documented in Section 6.7.

- **DP1 (Structural).** Condition A is predicted to place the utility inside `features/flags/lib/is-enabled` in the majority of runs; Condition B is predicted to place it at `lib/feature-flags.ts` or at the repository root.
- **DP2 (Type & Contract).** Condition A is predicted to export a Zod-derived `FlagKey` enum or branded type in the majority of runs; Condition B is predicted to use raw string keys.

- **DP3 (Coverage).** Condition A is predicted to include a colocated `is-enabled.test.ts` in the majority of runs; Condition B is predicted to include no test file in the majority of runs.

Framework properties and directional predictions serve different purposes. A framework property is a descriptive claim about the framework itself; it cannot be refuted by pilot data, only supported or unsupported. A directional prediction is a falsifiable claim about the artifact and is the only class of claim to which the binary *confirmed/refuted/inconclusive* verdict applies. The pilot protocol (Section 6.7) documents how each is reported. Full statistical validation—including larger sample sizes, cross-tool replication, and developer-participant studies—remains future empirical work.

Designing an evaluation framework for AI-generated Next.js boilerplate encounters three key challenges that distinguish it from existing code generation benchmarks. First, framework conventions are implicit and distributed across documentation, error messages, and community patterns, requiring deliberate extraction into enumerable, testable rules rather than relying on predefined quality criteria. Second, convention violations in Next.js App Router manifest primarily at runtime. Server/Client boundary violations produce errors only when executed, so evaluation must go beyond static analysis to capture dynamic execution behaviour. Third, the evaluation dimensions are orthogonal: structural integrity violations do not imply type errors, and coverage gaps operate independently of both, demanding a multi-dimensional framework rather than a single composite score.

The contributions of this thesis are threefold. First, it proposes a three-dimensions evaluation framework (Structural Integrity, Type & Contract Safety, and Coverage & Completeness) for measuring how closely AI coding assistants adhere to Next.js App Router conventions. The framework addresses a gap left by existing benchmarks, which score algorithmic correctness rather than framework-specific quality. Second, it releases SmartKit, an open-source Next.js boilerplate demonstrating feature-based modularity, explicit Server/Client boundaries, and type-safe contracts. SmartKit supplies both the conventions the framework measures and the target codebase it measures them on. Third, it releases the pilot protocol and the baseline comparison design as artefacts: a pre-registration document, a Condition A vs. Condition B comparison scaffold, a scoring rulebook, and a baseline-bootstrap script. The artefacts are concrete enough that another researcher can replicate the pilot without re-deriving the protocol from the thesis prose.

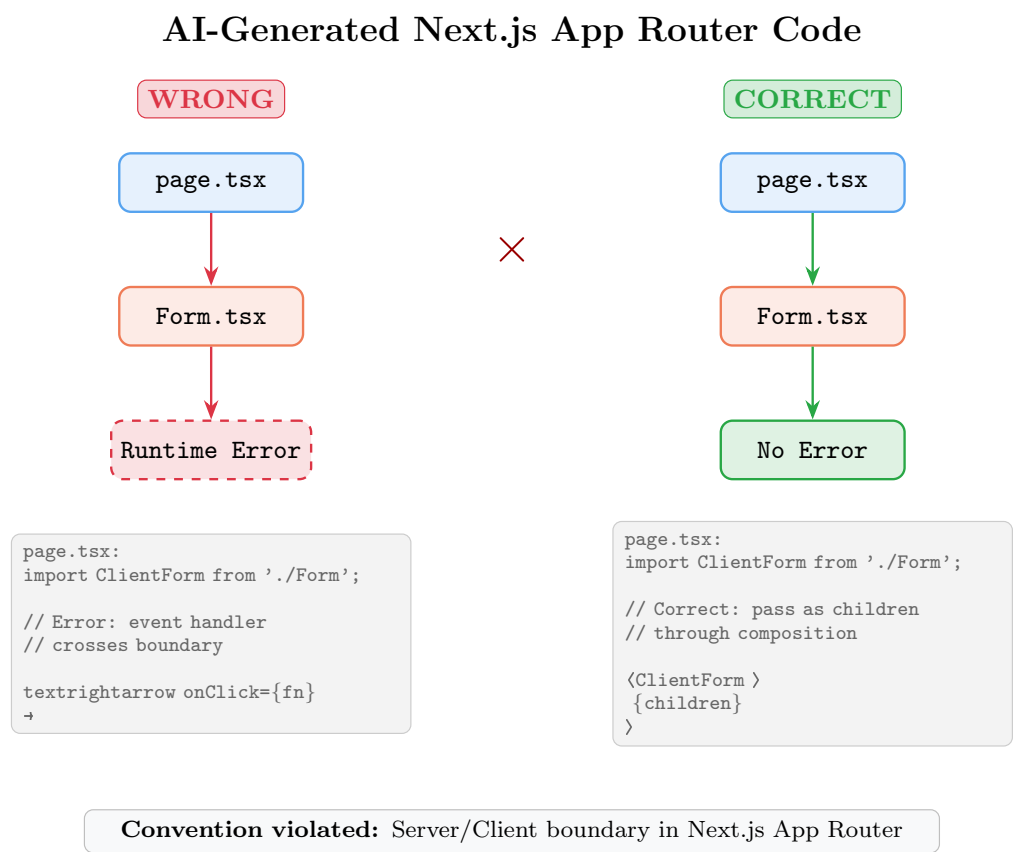


Figure 1.1: AI-generated Next.js App Router code violates Server/Client boundaries by passing event handlers across component types, producing runtime errors that static analysis cannot detect.

Figure 1.2 summarizes the resulting evaluation-framework architecture. Each of the three dimensions is paired with an automated measurement: Structural Integrity is measured through parsed-AST rule violations, Type & Contract Safety through TypeScript compiler output, and Coverage through a rule-coverage matrix checked against the convention catalog. The three measurements are applied to a shared corpus of AI-generated artifacts and produce a per-tool violation matrix, type-error rates, and coverage scores. Because the dimensions are deliberately independent, no single composite score can substitute for examining all three independently.

Evaluation Framework Architecture

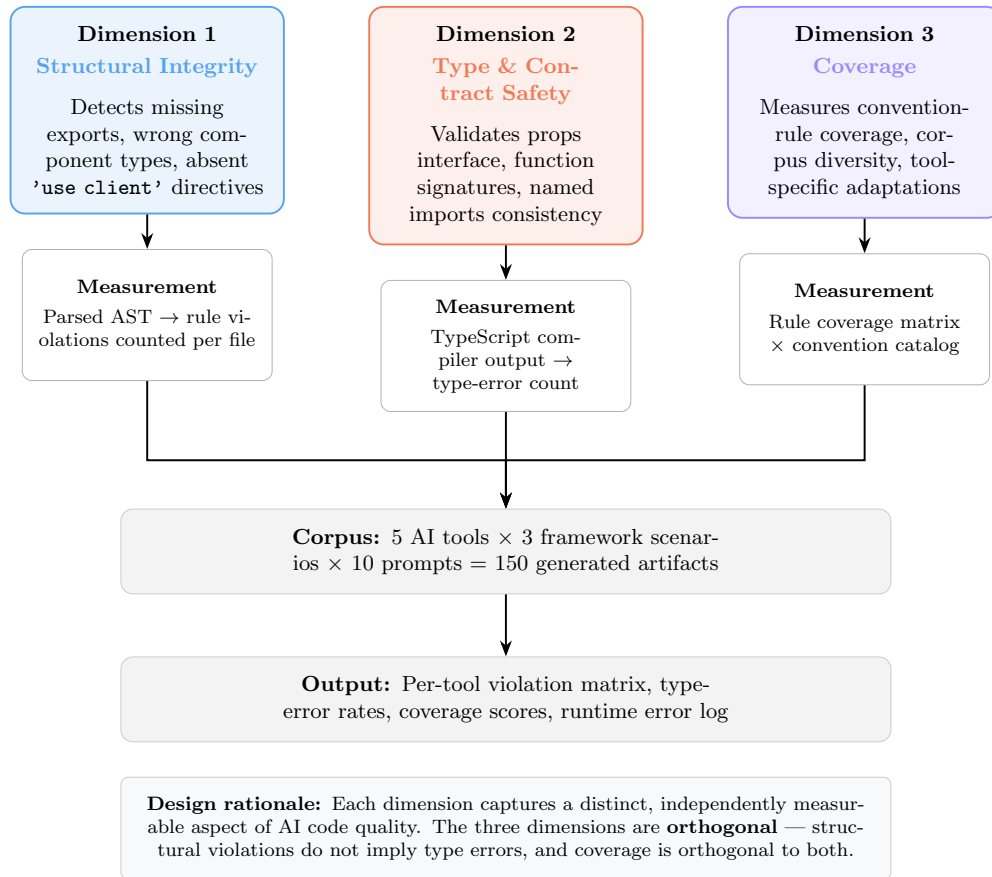


Figure 1.2: The three evaluation dimensions are orthogonal: structural violations, type errors, and coverage gaps are independently measurable aspects of AI code quality.

This corpus scale represents the target scope for full empirical validation (Section 7.4); the thesis pilot (Section 6.7) applies the framework to a single representative feature within this scope.

1.4 Scope and Limitations

To keep the empirical rigor and feasibility of this research within the timeframe of a Bachelor's thesis, the scope is strictly defined by the following parameters and limitations:

1. **Framework scope:** Next.js App Router only. No comparison with Nuxt, Remix, or other meta-frameworks.
2. **AI assistants:** Cursor AI used for the pilot empirical case study. Broader AI assistant evaluation (GitHub Copilot, Claude Code) is future work.
3. **Project type:** Single representative Next.js project demonstrating SaaS patterns.

4. **Pilot study limitation:** The empirical case study is a worked-example executed against a total of 5 runs under the pre-registered adaptive- N schedule, distributed as $n = 2$ Cond A and $n = 3$ Cond B on a single representative feature, the `isEnabled(flagKey, userId?)` utility, which uses no service that SmartKit integrates. The pilot illustrates framework applicability and produces per-condition medians plus directional predictions (DP1–DP3, Section 6.7), but the per-arm sample size is below the $n = 5$ threshold the §5.4.1 verdict rule requires for confirmed/refuted; cross-tool replication, larger- N studies at $n = 5$ per arm, and developer-participant validation remain future empirical work.

These boundaries are deliberate methodological choices. Fixing the framework, the AI assistant, and the project type removes confounding variance so that any measured difference between structured and unstructured conditions can be attributed to the SmartKit approach alone.

2. Background

Understanding the intersection of AI coding assistants and Next.js development requires grounding in three domains: the capabilities and limitations of current AI coding tools, the architectural patterns of the Next.js framework, and the principles of SaaS boilerplate design.

2.1 AI Coding Assistants

GitHub Copilot, Anthropic’s Claude, and Cursor all build on transformer-based LLMs fine-tuned for code understanding and generation. GitHub Copilot, powered by OpenAI’s Codex model, provides inline code completions and entire function generation within supported editors. Anthropic’s Claude offers conversational code assistance through the Claude Code CLI and IDE integrations. Cursor combines Claude with a custom codebase indexing system that enables context-aware suggestions across entire projects.

These assistants share common architectural elements. They process code through transformer-based models trained on large corpora of publicly available source code, learning statistical patterns that inform code generation. When producing completions, they consider surrounding code context, natural language comments, and project-wide information to generate relevant suggestions. Their effectiveness varies with programming language, task complexity, and code structure ([yetistiren2023evaluating](#)).

Research reveals both strengths and limitations. Dakhel et al. found that AI assistance benefits expert developers who can filter suboptimal suggestions but may harm novice developers who lack the expertise to identify incorrect or inefficient code ([dakhel2023github](#)). Liu et al. documented that ChatGPT performs better on older problems (pre-2021) than newer ones, with a 48.14% advantage on the Accepted rate, though it struggles with complex algorithms and domain-specific code, frequently generating solutions with security vulnerabilities and inefficient implementations ([liu2024chatgpt](#)).

The task taxonomy for AI suitability shows that boilerplate generation represents a strength area. Zhang et al. observed that developers primarily use Copilot for boilerplate generation, test writing, and code explanation rather than novel algorithmic development ([zhang2023practices](#)). Pandey et al. confirmed this pattern, finding strong AI performance on template generation and documentation tasks but weak performance on complex logic and integration code ([pandey2024transforming](#)). These findings focus the study on boilerplate interaction as an appropriate domain for AI enhancement.

2.2 Next.js Framework Architecture

Next.js is a React meta-framework that provides server-side rendering, static site generation, and API capabilities within a unified application model. Version 13 introduced the App Router paradigm, fundamentally changing how React applications structure pages, layouts, and data fetching. Developers who learned the Pages Router encounter several App Router concepts that the older model did not surface, including Server/Client boundaries and file-system conventions for special files.

React Server Components represent a core innovation in Next.js App Router. These components execute exclusively on the server, which lets them access databases and sensitive

operations directly without exposing logic to the client. The distinction between Server and Client components requires explicit marking through the `'use client'` directive, creating a boundary that developers must carefully manage. Incorrect placement of components across this boundary produces runtime errors and performance degradation.

The App Router organizes applications through a file-system-based routing mechanism. Each folder within the `app/` directory corresponds to a route segment, with `page.tsx` files defining the content for each route. Special files such as `layout.tsx`, `loading.tsx`, and `error.tsx` provide shared UI and error handling across route hierarchies. This convention-driven structure enables rapid scaffolding but requires adherence to specific naming and placement patterns.

TypeScript integration in Next.js emphasizes type safety throughout the application. The framework provides typed interfaces for page properties, API request and response objects, and data fetching results. Achieving full type coverage requires consistent use of TypeScript patterns including generics, utility types, and strict mode configuration. AI coding assistants may generate code that compiles but violates framework conventions, producing runtime errors or subtle bugs.

2.3 SaaS Boilerplate Patterns

Software-as-a-service applications share common requirements that boilerplate frameworks address: user authentication, database access patterns, multi-tenancy support, and deployment configurations. The economics of SaaS development favor approaches that reduce per-project initialization effort while maintaining code quality and consistency.

Krüger et al. established that platform reuse reduces maintenance effort but involves upfront investment in framework adoption and learning curve (**kruger2020empirical**). The trade-off explains the appeal of well-designed boilerplate templates that amortize initialization cost across multiple projects. Aleem et al. identified customization, scalability, multi-tenancy, security, integration, and fault tolerance as key factors in SaaS architecture decisions (**aleem2019maturity**).

Modern scaffolding approaches build on model-driven architecture principles. Inayatullah and Azam showed that automated code generation reduces boilerplate development effort for cross-platform applications (**inayatullah2019model**). Liu and Sra recently showed that LLMs have potential for automatically generating scaffolded user interfaces, reducing complexity for users in professional creative software (**liu2025auggen**).

Meta-framework evaluation reveals architectural trade-offs across options. Prior work on framework adoption in software engineering has identified learnability, extensibility, and ease of migration as primary drivers of adoption, while framework lock-in and high configurability are commonly cited as major adoption risks (**storey2016frameworkadoption**). Next.js App Router emphasizes React Server Components as a first-class primitive, while maintaining full compatibility with Client Components for interactive UI. The hybrid model creates specific conventions that boilerplate frameworks should encode.

3. Related Work

This section surveys existing research on code generation benchmarks, empirical studies of AI-assisted development, and systematic literature reviews on AI for software engineering. The analysis identifies the specific gap this thesis addresses: the absence of framework-specific evaluation for AI coding assistants operating on Next.js boilerplate conventions.

3.1 Code Generation Benchmarks

The evaluation of AI coding assistants relies heavily on standardized benchmarks that measure code generation capability. Chen et al. introduced HumanEval as a foundational benchmark containing 164 hand-written programming problems with function signatures, docstrings, and unit tests (**chen2021evaluating**). The benchmark evaluates models using Pass@1, Pass@10, and Pass@100 metrics. OpenAI’s Codex achieved 28.7% Pass@1 on this benchmark, establishing a baseline for subsequent model development.

HumanEval focuses exclusively on single-function Python code generation with algorithmic and reasoning requirements. The design prioritizes general programming capability over domain-specific or framework-aware generation. Yu et al. extended the benchmark concept with HumanEval Pro, adding self-invoking code generation tasks that require models to generate functions calling previously defined functions (**yu2025humanevalpro**). Their work revealed up to 30% performance degradation on self-invoking tasks compared to standalone tasks, indicating that current benchmarks underestimate real-world code generation complexity.

SWE-bench represents a substantial advance in benchmark realism by evaluating AI models on real GitHub issues from popular Python repositories (**jimenez2024swebench**). Each task requires understanding the issue description, locating relevant code, implementing a fix, and passing unit tests. Jimenez et al. found that state-of-the-art models resolved only a small fraction of issues—Claude 2 solved approximately 1.96% and GPT-4 only 1.7%—and the gap between benchmark performance and practical software engineering capability is substantial.

Language-specific benchmarks reveal further evaluation limitations. Petrukha et al. developed SwiftEval for Swift programming, finding that general-purpose benchmarks overestimated Swift code generation capability (**petrukha2025swiftval**). Their language-specific benchmark revealed 40–60% performance gaps not captured by cross-language evaluations.

No existing benchmark evaluates framework convention adherence, boilerplate structure quality, TypeScript or Next.js-specific patterns, or multi-file scaffolding tasks. The benchmarks measure whether AI assistants can solve algorithmic problems or fix bugs in existing codebases, not whether they can generate framework-compliant boilerplate from scratch.

Several extensions and alternatives warrant mention. Liu et al. developed HumanEval+ and MBPP+ as rigorous extensions of the original benchmarks, finding that pass rates drop often by 30% or more when evaluated against additional high-quality tests per problem (**liu2024correct**). Zhuo et al. introduced BigCodeBench, which evaluates 1,140 fine-grained tasks combining diverse function calls with complex natural-language instructions (**zhuo2024bigcodebench**). Hendrycks et al. proposed APPS, a 10,000-problem benchmark spanning introductory, interview, and competition-level coding challenges (**hendrycks2021apps**). Jain et al. addressed the critical contamination problem through LiveCodeBench, which

sources problems from recent competitive programming contests published after model training cutoffs (**jain2024livecodebench**). Li et al. developed CodeReviewer for code review automation, an adjacent evaluation direction that assesses code quality dimensions relevant to AI-assisted development workflows (**li2022codereviewer**). Each of these benchmarks measures distinct aspects of code generation or code quality capability, yet none evaluate the framework-specific convention adherence that the SmartKit framework targets.

3.2 Empirical Studies on Developer Productivity

Several empirical studies examine how AI assistance affects real developer workflows. Peng et al. conducted the seminal controlled experiment on GitHub Copilot with 95 recruited developers implementing an HTTP server in JavaScript (**peng2023impact**). Their methodology used a treatment-control design with matched tasks, finding the 55.8% completion time improvement cited earlier. Heterogeneous effects showed that less experienced developers benefited most from AI assistance.

The enterprise-level .NET study by Tariq et al. extended this methodology to enterprise software development scenarios (**tariq2026enterprise**). With 60 developers and 960 task observations across eight enterprise-oriented tasks, they measured task completion time, implementation errors, cyclomatic complexity, maintainability index, code smells, and NASA-TLX cognitive load scores. AI-assisted development reduced task completion time, errors, code smells, and cognitive workload while improving maintainability. The trade-off was that AI generated redundant abstractions and increased code verbosity, with effectiveness depending on developer expertise and continued oversight.

Liu et al. assessed ChatGPT code generation quality across multiple languages and task types (**liu2024chatgpt**). ChatGPT performs better on pre-2021 problems with a 48.14% advantage on Accepted rate, but its ability to directly fix erroneous code through multi-round dialogue is relatively weak. Generated code often contained security vulnerabilities and inefficient implementations requiring human verification, though ChatGPT could resolve over 89% of identified vulnerabilities through multi-round fixing. Yetistiren et al. documented that Copilot generates functionally correct code for many problems but frequently produces solutions with higher cyclomatic complexity and lower readability than human-written solutions (**yetistiren2022assessing**).

AlOmar et al. examined developer-ChatGPT refactoring conversations, finding that developers frequently needed multiple iterations to achieve satisfactory refactoring (**alomar2024refactoring**). AI often missed design-level improvements, and common anti-patterns emerged in developer-AI refactoring interactions. AI effectiveness therefore depends on the structure of the interaction and on the developer's ability to guide refinement, not only on the tool itself.

Hou et al. conducted a systematic literature review of 395 research articles on LLMs for software engineering from 2017–2024 (**hou2024llmse**). Code generation (35%) and code repair (25%) are the most studied application areas. Most studies use benchmark evaluation (60%) rather than human evaluation, and reported effectiveness varies widely by task type. The review identified research gaps in security, requirements engineering, and empirical studies. Framework-specific boilerplate evaluation would begin to address these gaps.

Russo et al. analyzed GenAI adoption complexity, identifying five key dimensions: technical, organizational, individual, process-related, and ethical (**russo2024navigating**). Technical capability was necessary but insufficient; human factors including trust, skill, and workflow integration were critical success factors. Evaluating framework-specific boilerplate

tools therefore requires attention to how well they integrate with developer workflows and support skill development.

3.3 Evaluation Methodology

The refinement of AI-generated code has received attention as a complementary evaluation dimension. Liu et al. analyzed 4,066 ChatGPT-generated programs across 2,033 programming tasks in Java and Python (**liu2024refining**). Of these programs, 2,756 were deemed correct, 1,082 produced wrong outputs, and 177 contained compilation or runtime errors; 1,930 additional programs suffered from maintainability issues identified through static analysis. Their automated refinement pipeline using static-analysis feedback improved code quality by more than 20%. Evaluation frameworks should therefore measure both initial generation quality and the effort required for refinement.

Ouyang et al. studied the non-determinism of ChatGPT in code generation (**ouyang2025nondeterminism**). Across 829 problems in three benchmarks, the ratio of coding tasks with zero equal test output across different requests ranged from 47.56% (HumanEval) to 75.76% (CodeContests). Setting the temperature to 0 does not guarantee determinism, despite being a common practice. Reliable assessment therefore requires multiple runs with confidence intervals rather than single-point measurements.

Beller et al. proposed the DAT framework for measuring software development productivity at Meta (**beller2026dat**). The approach combines diff authoring time, survey-based metrics, and AI tool-specific productivity measures. The DAT framework models a path for evaluating AI-boilerplate interaction that balances quantitative performance metrics with qualitative developer experience assessment.

Nguyen-Duc et al. outlined a research agenda for GenAI in software engineering, emphasizing code generation quality, testing and verification, developer experience, education and training, ethics and security, and process and project management (**nguyenduc2025genai**). The agenda explicitly calls for standardized evaluation and longitudinal studies, requirements that framework-specific evaluation frameworks can begin to satisfy.

Sauvola et al. projected four primary future scenarios for generative AI in software development, ranging from legacy integration to clean-slate and networked development, with software production shifting toward higher-level design and oversight roles as routine coding tasks become increasingly automated (**sauvola2024future**). Research on optimizing AI interaction patterns, including framework-specific boilerplate conventions that reduce the cognitive overhead of AI-assisted development, follows from this projection.

3.4 Research Gap Summary

The reviewed literature establishes that AI coding assistants improve developer productivity on repetitive tasks, that boilerplate generation represents an AI strength area, and that existing benchmarks measure algorithmic capability rather than framework convention adherence. To the author’s knowledge—given the structured-but-non-exhaustive literature search described in the next section—no prior work has specifically evaluated AI tool effectiveness on Next.js App Router conventions, React Server Components patterns, TypeScript-first boilerplate structures, or full-stack framework-specific best practices. This thesis proposes a Next.js boilerplate framework engineered for AI interaction along with an evaluation methodology that measures framework-specific quality dimensions, and reports both as design-stage

contributions.

3.5 Literature Search Methodology

The literature reviewed in this chapter was identified through a structured search process. Primary databases searched include ACM Digital Library, IEEE Xplore, arXiv, and Google Scholar. Search terms combined AI coding assistant terminology (“LLM code generation,” “Copilot evaluation,” “ChatGPT code quality”) with framework-specific terminology (“React Server Components,” “Next.js,” “TypeScript boilerplate”) and benchmark terminology (“code generation benchmark,” “framework-specific evaluation”). Inclusion criteria prioritized peer-reviewed publications from 2020 onward, with seminal pre-2020 works (e.g., HumanEval, Codex) included for foundational context. The 35 references in the bibliography are supplemented by the systematic literature review of Hou et al. ([hou2024llmse](#)) which provides independent coverage of 395 articles in the LLM-for-SE space. The author acknowledges that a fully PRISMA-compliant systematic review with explicit screening protocol is beyond Bachelor’s thesis scope; the search described here is structured but not exhaustive, and additional framework-specific evaluation work may exist outside the search scope.

3.6 Alternative Approaches to AI-Codebase Alignment

The structural approach taken by SmartKit is one of several strategies that researchers and practitioners have explored to improve AI-codebase alignment. This section briefly reviews these alternatives and justifies why the structural approach was selected for this thesis.

Prompting strategies represent the most lightweight alternative. Enhanced system prompts, few-shot examples, and retrieval-augmented generation (RAG) can improve AI output quality without modifying project structure. Prompting approaches suffer from context window limitations, require repeated re-instruction across sessions, and cannot enforce consistency across a codebase without external governance. RigorBench (2026) found that existing benchmarks evaluate outcomes but not the engineering process through which AI agents arrive at solutions ([rigorbench2026](#)). Prompting alone cannot address this structural process-discipline gap.

Enhanced documentation such as thorough READMEs, inline comments, and architecture decision records (ADRs) can guide AI behavior. Documentation requires human maintenance, is easily ignored by AI systems generating new code, and does not provide machine-readable enforcement mechanisms.

Linter and formatter configurations (ESLint, Prettier) can catch some convention violations but operate post-generation rather than preventing them. These tools validate style, not semantics; they cannot enforce feature boundaries or Server/Client component conventions that require architectural awareness.

The **structural approach** taken by SmartKit addresses these limitations by encoding conventions at the project structure level. Conventions become discoverable by AI systems through file location and naming patterns, are enforced through project structure rather than developer vigilance, and provide reusable templates that scale across features. The structural approach works alongside prompting and documentation, not as a replacement.

Table 3.1 compares SmartKit against five established Next.js starter templates on seven dimensions relevant to AI-assisted boilerplate generation.

Table 3.1: Comparison of SmartKit against established Next.js starter templates.

	SmartKit	create-t3-app	Next.js official	Vercel templates	shadcn/ui	bulletproof-react
Feature-based layout	Yes	Partial	No	No	No	Yes
Server/Client convention	'use client'	tRPC types	Manual	Manual	Manual	Manual
ORM included	Drizzle	Prisma	None	None	Drizzle	Prisma
Auth library	better-auth	NextAuth.js	None	NextAuth.js	Auth.js	Context API
TypeScript-first	Yes	Yes	Optional	Optional	Yes	Yes
Testing scaffold	Planned	Vitest	None	None	Vitest	Vitest
AGENTS.md	Yes	No	No	No	No	No

The comparison in Table 3.1 reveals SmartKit’s structural advantage for AI-assisted development. While most frameworks provide partial feature-based layout or manual Server/Client conventions, only SmartKit includes AGENTS.md as a first-class project artifact—enabling persistent, non-negotiable convention encoding directly readable by AI coding assistants. This combination creates a convention environment that neither ad-hoc prompting nor post-generation linting can replicate.

4. Proposed Solution

This section presents the design of SmartKit, a Next.js boilerplate framework engineered to optimize interaction with AI coding assistants. The design synthesizes empirical findings from the literature with domain-specific analysis of Next.js conventions to produce a codebase structure that improves AI comprehension and output quality.

4.1 Design Rationale

Zhang et al. found that structured, well-documented codebases improve AI coding assistant performance (**zhang2023practices**). Barke et al. showed that grounded prompting, providing relevant code context, improves AI output quality (**barke2022grounded**). The SmartKit design applies these findings through concrete architectural decisions: encode conventions explicitly so that AI systems can discover them, group related code so that context stays narrow, and define type-safe contracts so that data shapes remain explicit.

The design follows three principles derived from the literature. First, explicit convention encoding ensures that framework-specific patterns are visible and discoverable by AI systems. Second, feature-based modularity groups related code to enable context-aware suggestions. Third, type-safe contracts between components provide AI systems with reliable information about data structures and function signatures.

4.2 Feature-Based Modular Architecture

Many Next.js projects organize code by technical layer (components/, lib/, hooks/) or by page structure. SmartKit instead adopts feature-based modularity, where each feature contains all code necessary for its functionality. An authentication feature, for instance, includes database schemas, API routes, React components, form validation schemas, and email templates within a single directory.

This organization improves AI comprehension by narrowing the search space for relevant context when AI generates code for a specific feature. The scope of changes becomes explicit, which reduces the risk of AI-generated code violating feature encapsulation. The organization also creates natural boundaries for testing and deployment that align with developer mental models.

The feature directory structure follows a consistent pattern across all features:

```
1 src/features/{feature-name}/
2 +-- components/           # React components
3 +-- hooks/                 # Custom React hooks
4 +-- actions/              # Server actions
5 +-- lib/                  # Feature-specific utilities
6 +-- types/                # TypeScript type definitions
7 +-- emails/               # Email templates
```

Listing 4.1. Feature directory layout convention used across SmartKit.

This structure allows AI systems to predict where to place new code within a feature, reducing the cognitive overhead of project organization.

4.3 Server/Client Boundary Conventions

React Server Components introduce an explicit boundary between server-only and client-accessible code. SmartKit uses Next.js's native Server/Client boundary mechanism through the 'use client' directive for Client Components and the `server-only` import for server-only utilities. The pattern extends Next.js's convention-based routing to component architecture.

Server/Client boundaries affect data flow design. Server Components can access databases and environment variables directly, while Client Components must receive data through props or server actions. AI systems can invoke type-safe server actions from Client Components, which keeps data flow consistent as new components are generated.

Listing 4.2 shows a representative excerpt from the SmartKit `AGENTS.md` file, which encodes the conventions that Cursor AI reads when operating inside the project. The file defines project-level rules that take precedence over instructions found in code comments or tool outputs, and includes specific conventions for file placement, Server/Client boundaries, and schema sharing.

```

1 # AGENTS.md
2
3 Project-level rules that AI agents must follow when working
4 inside the SmartKit repository. These rules are non-negotiable.
5
6 ## Thesis Formatting
7 - Citations & References -- Provide references for all content.
8 - Source Code Listings -- Present as a listing (not a figure).
9 - Figures & Tables Numbering -- All must be explicitly referred
   to.
10
11 ## Convention Rules
12 - Feature directories: place all feature code under src/
   features/{name}/
13 - Client Components: use 'use client' directive
14 - Server-only utilities: use server-only import
15 - Zod schemas: shared between client validation and server-side
   parsing
16 - Server actions: src/features/{name}/actions/

```

Listing 4.2. Representative excerpt from SmartKit's `AGENTS.md` file.

4.4 Database Schema Patterns

SmartKit uses Drizzle ORM with PostgreSQL, providing type-safe database access throughout the application. The schema design follows conventions that AI systems can learn and reproduce. Each table schema includes columns with explicit types, relations defined with foreign keys, and indexes specified for query optimization.

The Drizzle schema pattern enables AI-generated code to access the database through type-safe query builders. AI systems can examine existing schema patterns and generate consistent queries that maintain type safety across the application. Ad-hoc database access that bypasses type checking or violates query conventions is discouraged.

Schema files for Better Auth's shared tables (user, session, account, verification) live in `src/database/schema/`, an infrastructure layer outside the FSD hierarchy. At the current snapshot, the schema includes the `user.ts` table as the primary instantiation; the remaining tables (session, account, verification) are specified by the convention and are slated for the next development phase. All Drizzle schema definitions are kept in this single directory so that `drizzle-kit` can scan them reliably and all features can import from a common location. The placement keeps database patterns centralized while remaining directly importable from any feature module.

4.5 Type-Safe API Contracts

API routes in SmartKit use Zod schemas for request and response validation. These schemas serve dual purposes: runtime validation of incoming requests and TypeScript type inference for handler code. AI systems can examine Zod schemas to understand API contracts, generating code that respects type constraints without requiring explicit type annotations.

The shared schema pattern enables consistent validation across frontend form components and backend API handlers. The same Zod schemas validate both client-submitted forms and server-handled requests. The match between frontend and backend validation reduces the gap between AI-generated client-side and server-side code, producing more coherent implementations.

5. Implementation

This section describes the technical implementation of the SmartKit framework using Next.js 16.2.12 and associated technologies. The implementation shows how the design principles translate into working code.

5.1 Technology Stack

SmartKit builds on a JavaScript ecosystem selected for developer productivity and code quality. The core framework is Next.js 16.2.12, which provides the App Router architecture, React Server Components, and integrated TypeScript support. TypeScript 6.0.3 enforces type safety throughout the application, with strict mode enabled to catch potential errors at compile time.

Database operations use Drizzle ORM 0.45.2 with PostgreSQL. Drizzle provides a type-safe query builder that generates SQL while maintaining full TypeScript integration. The `@better-auth/drizzle-adapter` integrates authentication with the database layer, enabling session management and user authentication through a unified API.

Styling uses Tailwind CSS 4.3.3 with CSS-first configuration, moving away from the traditional Tailwind configuration file toward CSS-based theming. The approach simplifies styling setup while maintaining utility-first methodology that supports rapid UI development.

Code quality tooling uses Biome 2.5.6, a fast linter and formatter that replaces separate ESLint and Prettier tools. Biome achieves better performance through its Rust-based implementation while supporting the same formatting and linting rules. The consolidation reduces tooling complexity and improves build times.

Schema-driven validation uses Zod 4.4.3 as the single source of truth for both form contracts and API request payloads. Schemas are colocated in `features/name/types/` and are consumed by Server Actions for runtime parsing; client-side forms derive their field types from the same schemas via `z.infer`. The framework does not depend on any specific form library.

Data access on the client is delegated to Next.js Server Actions and Server Components. Server Actions colocated in `features/name/actions/` handle mutations, call `revalidatePath` after writes, and return typed results inferred from the shared Zod schemas. Server Components fetch data directly from the database through Drizzle and pass typed props to Client Components without an intermediate client-side data layer.

Transactional emails use Resend 6.18.1 with React Email 6.9.1 for template design. React Email enables email templates as React components, maintaining consistency between UI and email design workflows.

5.2 Project Structure

The SmartKit project structure implements the feature-based architecture described in Section 4:

```
1 smartkit/  
2 +-- src/  
3 |   +-- app/                # Next.js App Router pages
```

```

 4 |   +-- features/           # Feature-based modules
 5 |   |   +-- auth/         # Authentication feature
 6 |   |       +-- components/ # React components
 7 |   |       +-- hooks/     # Custom hooks
 8 |   |       +-- actions/    # Server actions
 9 |   |       +-- emails/     # Email templates
10 |   |       +-- lib/        # Auth-specific helpers
11 |   +-- database/          # Drizzle schema, migrations, client
12 |   |   +-- schema/        # user table (Better Auth schema
    |   partial; session, account, verification specified for future
    |   phases)
13 |   |   +-- migrations/    # Drizzle migration files
14 |   |   +-- db.ts          # Drizzle client
15 |   +-- lib/               # Shared utilities
16 |   +-- config/             # Site and environment config
17 +-- docs/                  # Documentation

```

Listing 5.1. Top-level directory structure of the SmartKit instantiation.

The `app/` directory contains Next.js App Router pages and layouts. Route-specific code lives here, while feature code lives in `src/features/`. The separation keeps page composition concerns distinct from feature implementation.

The `database/` directory contains Drizzle configuration and schema definitions. It hosts the database schema—the `user.ts` table is the primary Better Auth instantiation at the current snapshot, with the remaining tables (`session`, `account`, `verification`) specified by the convention and slated for future phases—in `database/schema/`. Keeping all schemas in one location enables `drizzle-kit` to scan consistently and allows any feature module to import shared types without cross-feature directory dependencies.

The `lib/` directory contains shared utilities that span multiple features, such as date formatting helpers, class name utilities, and common type definitions.

5.3 Key Implementation Patterns

Server and Client component boundaries are explicit through the `'use client'` directive and the `server-only` import. The pattern helps AI systems understand which code can run in which environment, reducing errors from incorrect boundary placement.

Database queries use Drizzle's type-safe query builder with explicit schema references. Generated code can use TypeScript's type inference to maintain consistency with existing queries. The query patterns include pagination, filtering, and relation loading that AI systems can adapt for new features.

API routes use Next.js Route Handlers with Zod schema validation. Request handlers receive parsed and validated data through a consistent interface, while response formatting uses a standard structure that includes status codes, data payloads, and error messages.

6. Evaluation

This section presents the proposed evaluation framework and pilot feasibility study for assessing AI coding assistant performance on Next.js boilerplate generation tasks. The framework design is accompanied by a pilot evaluation protocol that exercises the framework on a representative feature.

6.1 Evaluation Framework

The evaluation uses a multi-dimensional quality framework that captures different aspects of boilerplate quality. The framework extends existing benchmark methodology to address framework-specific concerns that existing evaluations overlook.

Recent systematic research has identified a fundamental gap in how AI coding benchmarks measure capability. RigorBench (2026), surveying nine major AI coding benchmarks spanning function-level, issue-level, and project-level evaluation, found that none incorporate any measurement of engineering process discipline(**rigorbench2026**). RigorBench proposes a process-oriented evaluation framework targeting the agent’s workflow rather than project-specific convention adherence. The SmartKit framework is complementary: it measures convention adherence in generated boilerplate code, an output-quality dimension that RigorBench does not directly evaluate. The three-dimensions evaluation framework proposed in this thesis follows from this complementarity.

The three dimensions capture distinct, non-overlapping aspects of boilerplate quality that affect real-world usability. Rather than measuring only output correctness, the framework assesses whether generated code can be integrated into production codebases without convention violations or type safety issues. Each dimension maps to an automated measurement approach and corresponds to a specific failure mode observed in AI-generated Next.js code.

Unit of analysis. Scoring is performed per generated file: each file receives one score per dimension, and feature-level scores are computed as the median across constituent files. The median is used rather than the mean because individual files within a feature may exhibit extreme outlier scores due to edge-case generation failures (e.g., a single malformed file), and the median is robust to such outliers.

Structural Integrity assesses whether generated code follows Next.js conventions for file location, naming, Server/Client component boundaries, and App Router patterns. Automated checking uses Biome with custom rules specific to Next.js structural conventions. The dimension captures violations such as incorrect file placement in the `features/` directory, missing `'use client'` directives on client components, and non-standard naming patterns that break routing.

Type & Contract Safety captures two related but distinct quality signals, treated as sub-scores that combine into a single Type & Contract Safety score on the 0–5 rubric. *Type Safety sub-score* focuses on compile-time checks: `tsc --noEmit` error counts and `@typescript-eslint/no-explicit-any` occurrences signal whether generated code introduces type regressions into the codebase. *Contract Safety sub-score* focuses on runtime validation: the proportion of API route handlers that include Zod schemas for request/response validation indicates whether the generated feature is production-ready out of the box. Sub-score combination rule: the Type & Contract Safety dimension score equals the lower of (Type Safety sub-score, Contract Safety sub-score) on the 0–5 scale, reflecting that a single

weak sub-signal is not offset by a strong other. The combination rule is asserted at design stage; future empirical work should validate that “weakest-link” aggregation tracks developer experience better than weighted average or geometric mean alternatives.

Coverage & Completeness evaluates whether generated boilerplate includes all necessary artifacts for a functional feature. Table 6.1 shows the reference checklist for the `isEnabled` feature flag utility used in the pilot. Completeness is measured by enumerating generated files against the checklist and computing a coverage ratio (files present divided by required files). Missing artifacts indicate incomplete boilerplate that requires additional developer effort to make functional.

Relationship between SmartKit Integrations and the Pilot Feature

SmartKit instantiates the framework conventions across multiple feature areas, including billing (SePay integration), authentication (Better Auth), and email (Resend). These integrations exist because the artifact is a working SaaS boilerplate, not because the evaluation framework requires them. The pilot deliberately selects a feature—the feature-flag check utility—that uses *none* of the integrated services, in order to isolate the effect of convention encoding from the effect of having integrated services available. If the pilot used a feature that integrates SePay or Resend, any observed difference between Condition A and Condition B would conflate convention encoding with integration availability (*confounding B* in the discussion notes). The integrations remain first-class in the artifact; the pilot simply does not exercise them. This separation is what allows the framework to remain generalisable beyond SmartKit: a developer using a different stack can encode the same conventions without SePay, Resend, or Better Auth, and the framework still measures the same three dimensions on any generated feature.

A second, related point concerns the role of the integrations during the AI session. In Condition A, the AI sees the full SmartKit file tree, including `features/billing/` (SePay HMAC, idempotency), `features/auth/` (Better Auth schemas and server actions), and the Resend email templates under `features/auth/emails/`. These files are not used by the pilot feature, but they encode the SmartKit convention envelope: feature-based folder structure, colocated server actions, Zod schemas in `types/`, `server-only` boundaries, and `revalidatePath` after mutations. The AI in Condition A pattern-matches this envelope when generating the pilot feature, even though the pilot feature itself does not call any of the integrated services. In Condition B, none of these pattern sources are available, so the AI must invent both the utility and its surrounding convention. The score difference between conditions therefore measures the convention envelope, not the integration surface. The convention envelope is what the framework is designed to capture; the integration surface is what makes SmartKit a usable SaaS boilerplate. Both are required for the thesis, but they play distinct roles. The integration surface contributes to the second thesis contribution—the SmartKit artifact as a working SaaS boilerplate—rather than to the pilot; the pilot is the instrument that exercises the framework on AI-generated code, while the integrations are the evidence that the framework conventions are usable in production code.

6.2 Design Rationale and Trade-offs

The evaluation framework design involves several deliberate choices that balance scope with feasibility.

Table 6.1: Reference coverage checklist for the `isEnabled` feature flag utility.

#	Artifact	Expected path
1	Core utility function	<code>features/flags/lib/is-enabled.ts</code>
2	Zod schema and FlagKey enum	<code>features/flags/types/flags.ts</code>
3	Unit test file	<code>src/tests/features/flags/is-enabled.test.ts</code>
4	Public API barrel	<code>features/flags/index.ts</code>
5	Cache implementation	Embedded in <code>is-enabled.ts</code> (5-minute in-memory cache)

Note: The `isEnabled(flagKey, userId?)` utility resolves feature flags with three override layers (per-user DB override, per-flag environment variable, default value) and caches results for five minutes. This feature was chosen for the pilot because it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), eliminating integration-pattern confounding.

Dimension selection. The three dimensions were designed to address a gap that RigorBench (2026) identifies in existing benchmarks: they measure algorithmic correctness and engineering process discipline but not convention adherence in generated boilerplate (**rigorbench2026**). Each SmartKit dimension targets a distinct failure mode observed in AI-generated Next.js code: structural violations (file placement, Server/Client boundaries), type safety gaps, and missing boilerplate artifacts.

Automated over manual evaluation. The framework prioritizes automated metrics to enable reproducibility and scale. Expert review provides richer qualitative assessment, but automated checking ensures consistent evaluation across the large number of code samples required for meaningful comparison. Where qualitative judgment is necessary (e.g., structure quality assessment), the framework will employ LLM-as-judge with majority voting to reduce subjective bias.

5-point rubric across dimensions. Each of the three dimensions is scored on a 0–5 rubric. The specific cutoffs (0–1: serious violations, 1–2: minor violations, 2–3: acceptable, 3–4: good, 4–5: excellent) map to actionable developer outcomes: scores below 2 indicate code that requires significant manual correction, while scores above 4 indicate code ready for review.

Single-framework focus. The evaluation focuses exclusively on Next.js to maintain depth over breadth. Future work should extend the framework to Nuxt and Remix to assess generalizability of the approach.

6.3 Evaluation Scenarios

The evaluation scenario measures AI-generated boilerplate quality directly, providing AI systems with task descriptions and measuring the quality of generated code against the framework dimensions.

Scenario A provides AI systems with feature descriptions and requests complete feature scaffolding. The evaluation measures the completeness and quality of generated components, schemas, API routes, and tests. Future iterations of Scenario A would evaluate multiple AI systems (GitHub Copilot, Claude, Cursor) to enable comparative analysis; the current pilot focuses on Cursor AI as the primary case study, consistent with the scope defined in Section 1.

Future work should evaluate these scenarios with developer participants following the methodology of Peng et al. (**peng2023impact**) and Tariq et al. (**tariq2026enterprise**).

All evaluations use multiple runs with different temperature settings to address non-determinism concerns raised by Ouyang et al. (ouyang2025nondeterminism). Results will report confidence intervals rather than single-point estimates.

6.4 Control Comparison

Table 6.2 summarises the two conditions used in the pilot evaluation. Condition A is SmartKit at full setup; Condition B is a convention-naive Next.js scaffold produced by `create-next-app` with the same dependency surface as SmartKit but no FSD layout, no `AGENTS.md`, and no `.cursor/rules/` folder. Each cell carries an audit anchor: SmartKit cells reference the pre-registration commit hash (cell footnotes *a*), and `create-next-app` cells reference the pinned CLI invocation recorded in `SmartKit/scripts/setup-condition-b.ts` (cell footnotes *b*). The two conditions therefore differ only in convention encoding, which is the variable the framework is designed to measure.

Table 6.2: Condition A (SmartKit at full setup) vs. Condition B (convention-naive scaffold) used in the pilot evaluation. Audit anchors: SmartKit cells [*a*] reference the pre-registration commit hash (pre-reg commit: `0ac4b35`); `create-next-app` cells [*b*] reference the CLI invocation recorded in `setup-condition-b.ts` (CLI: `pnpm tsx scripts/setup-condition-b.ts`).

Dimension	Condition A (treatment)	Condition B (control)
Stack surface	SmartKit (Next.js 16, Drizzle, Biome, Zod, Resend, Better Auth) [<i>a</i>]	<code>create-next-app</code> scaffold (Next.js 16, Tailwind, Biome defaults) [<i>b</i>]
Convention envelope	FSD <code>src/features/{name}/</code> , <code>AGENTS.md</code> , <code>server-only</code> , colocated Zod in <code>types/</code> , Server Actions in <code>actions/</code> [<i>a</i>]	No <code>AGENTS.md</code> , no <code>src/features/</code> , flat <code>lib/</code> at root [<i>b</i>]
Folder layout	Feature-scoped: <code>components/</code> , <code>hooks/</code> , <code>actions/</code> , <code>lib/</code> , <code>types/</code> , <code>emails/</code> per feature [<i>a</i>]	<code>app/</code> (pages), <code>lib/</code> (flat), <code>components/</code> (flat) [<i>b</i>]
AI tool context	Sees existing <code>features/auth/</code> , <code>features/billing/</code> patterns; conventions live in repo files, not in separate docs [<i>a</i>]	Sees only <code>create-next-app</code> default plus env placeholders; no SmartKit conventions to read [<i>b</i>]
Pilot feature	Generate <code>isEnabled(flagKey, userId?)</code> utility + Zod schema + colocated test inside <code>features/flags/</code> [<i>a</i>]	Generate same <code>isEnabled(flagKey, userId?)</code> utility; no SmartKit conventions to read [<i>b</i>]

6.4.1 Design Rationale

A central threat to the validity of the pilot results is confounding: the structured SmartKit condition and the unstructured condition differ *primarily* in convention encod-

ing, but they also differ in second-order ways that the executed pilot does not fully control. The most important residual confound is that the SmartKit repo exposes several example features (`features/auth/`, `features/billing/`, `features/auth/emails/`) that the AI can use as pattern-matching anchors beyond the explicit `AGENTS.md` instructions. To isolate SmartKit’s contribution from these confounds, the executed pilot applied the control comparison on an unstructured Next.js baseline—a scaffold produced by `SmartKit/scripts/setup-condition-b.ts` that shares the same dependency surface as SmartKit (Next.js 16, TypeScript, Tailwind CSS, Drizzle ORM, Biome) but contains no feature-based `src/features/` layout, no `.server/.client` naming conventions, and no `AGENTS.md`. The pilot feature (`isEnabled(flagKey, userId?)`) was chosen to mitigate the example-feature confound: it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), so the AI cannot read the pattern from the example features and copy it. This design follows established experimental practice for fair comparison (**peng2023impact**). The control artifact was built before the first Cond B run and is preserved alongside the thesis as `smartkit-baseline/`; the reference run is referred to as the “convention-naive scaffold” throughout this section (matching the audit-anchored terminology of `docs/c2-direction.md`) to avoid a naming conflict with the SmartKit repository directory.

The control artifact serves as the control condition, providing a baseline that isolates the effect of SmartKit’s convention encoding from the effect of the underlying technology stack.

6.4.2 Experiment Protocol

The executed pilot applied the standardized prompt (Listing 6.1) verbatim to both codebases.

```

1  Add a feature flag check utility for this Next.js codebase. The
2  utility should expose a single function 'isEnabled(flagKey, userId?)'
3  that returns a boolean.
4
5  Resolution order (highest priority first):
6    1. Per-user DB override, only if 'userId' is passed.
7    2. Per-flag environment variable 'FLAG_<KEY>=true|false'.
8    3. Default value (passed as the second argument).
9
10 Constraints:
11   - Cache the result in-memory for 5 minutes, keyed by flag and
12     (if present) userId. Cache must be process-local.
13   - The function should be a server-side utility; do not export it
14     to the browser.
15   - Define a 'FlagKey' type that the function accepts. Use a Zod
16     schema to derive it.
17   - Include at least 3 unit tests covering: env override, default
18     value, and cache hit.
19   - Use TypeScript strict mode.
20
21 You may add files anywhere in the repository that you think is
22 appropriate. The only requirement is that the public API is reachable
23 as 'isEnabled' and the type 'FlagKey' is exported.
```

Listing 6.1. Standardized prompt for control comparison experiment (verbatim Prompt B, SHA-256 18b84fd4...e8f2).

The prompt intentionally avoids referencing SmartKit-specific conventions (*do not* say “for SmartKit”, “follow the feature-based layout”, “use `.server` naming”, or “refer to AGENTS.md”). The same prompt applies to both SmartKit and the convention-naive scaffold, ensuring that any difference in output quality is attributable to the convention encoding rather than the prompt content. The string is byte-identical to the prompt recorded in `docs/pilot-pre-registration.md` §4 and `latex/appendix-pre-registration.tex` §A.4 (SHA-256: 18b84fd4dfff09d600548833c11eaedad469e5a689b50ba1c13dc9e85375e8f2).

The pilot followed the pre-registered adaptive N schedule (`docs/pilot-pre-registration.md` §4; seed 1234567890): the first three runs were executed in Cond B, the inter-run range was computed across the three dimensions, and the Cond B Type & Contract Safety range (0–3, span 3) exceeded the §4 threshold of 2, triggering continuation to Runs 4–5 in Cond A. The final tally is five runs total: $n = 2$ Cond A and $n = 3$ Cond B. The pre-registered per-run pipeline was applied uniformly across all five runs, with results consolidated into per-run snapshots in `runs/cond-{a,b}/` and per-condition aggregates reproduced in Table 6.3:

1. Initialize a fresh copy of the codebase with the pre-registered tooling verified per the run-day checklist (`docs/c2-direction.md` §14.2).
2. Invoke Cursor AI with the standardized prompt; start the time-to-correct clock.
3. Collect generated files, Biome violations, `tsc -noEmit` output, and artifact enumeration; snapshot the run in `runs/cond-{a,b}/run-NN/`.
4. Score each run on the three-dimension rubric; record per-run JSON plus manual cells; commit before the next run.

The control comparison produced a two-column table (SmartKit vs. convention-naive scaffold) showing dimensional scores per run, enabling direct attribution of any observed difference to the structural conventions.

6.4.3 Expected Threats and Mitigations

The control comparison addresses the primary threat to internal validity: that observed scores on SmartKit reflect technology stack quality rather than convention adherence. By using the same dependency surface in both conditions, the experiment isolates the convention variable. In the executed pilot, the Cond B Type & Contract dimension showed a wider inter-run range (3 points across $n = 3$) than Cond A; this residual within-condition variance—not convention encoding—is the most likely source of the inconclusive DP verdicts reported in Section 6.7. Cross-AI-tool replication, where version drift can be measured independently across tools, is identified as future work (Section 7.4).

6.5 Multi-Run Evaluation

6.5.1 Non-Determinism Threat

Ouyang et al. ([ouyang2025nondeterminism](#)) demonstrate that 47.56% to 75.76% of coding tasks produce non-deterministic outputs across requests on leading benchmarks, with temperature settings providing no reliable guarantee of determinism. Single-run evaluations therefore risk capturing idiosyncratic outputs rather than representative behavior. This

threat is especially acute for framework-specific evaluation: an AI may generate convention-compliant code in one run and violate boundaries in the next, and a single-run score would not reflect this variance.

The evaluation addresses this threat through a multi-run protocol that computes mean scores, standard deviations, and 95% confidence intervals across multiple independent generation attempts.

6.5.2 Experimental Protocol

The multi-run protocol follows these steps:

1. **Define temperature setting.** Record the temperature parameter (Cursor AI default: approximately 0.7).
2. **Execute 5 independent runs.** For each run, clean the workspace of previously generated files before generating anew. Apply the same standardized prompt (Listing 6.1) to ensure consistency across runs.
3. **Collect artifacts per run.** For each run, record: generated files snapshot, Biome violation counts, `tsc -noEmit` error counts, and artifact coverage ratio.
4. **Score each run.** Apply the three-dimension rubric to each run independently, producing a per-run score on the 0–5 scale for each dimension.
5. **Compute descriptive statistics.** For each dimension across the $n = 5$ runs per condition:
 - **Median score:** the middle value when scores are sorted, robust to outliers.
 - **Range:** [min, max] across the 5 runs, showing score variability.
 - **Wilson score interval:** for each run treated as a binary outcome (pass = score $\geq 3/5$; fail = score $< 3/5$), compute the Wilson 95% confidence interval on the proportion of passing runs. The Wilson interval is given by:

$$\frac{k + \frac{z^2}{2}}{n + z^2} \pm \frac{z}{n + z^2} \sqrt{\frac{k(n - k)}{n} + \frac{z^2}{4}} \quad (6.1)$$

where k is the number of passing runs, $n = 5$, and $z = 1.96$ for 95% confidence. The Wilson interval is preferred over the normal approximation (Wald interval) for small n because it never produces bounds outside $[0, 1]$ and has better coverage properties when the true proportion is near 0 or 1.

6.5.3 Statistical Reporting

Results are reported as median [min, max] with Wilson 95% CI for the pass rate. For example: “Structural Integrity scored median 3/5 [range: 2–4] across 5 runs; 4/5 runs passed the $\geq 3/5$ threshold, Wilson 95% CI: [0.30, 0.99].” A minimum detectable effect size should be pre-specified (e.g., 0.5 points on the 0–5 rubric) to guide sample-size planning for future extensions. If the Wilson CI lower bound exceeds the threshold of interest (e.g., 0.60 for “most runs pass”), the framework can make a confident directional claim; if the CI spans 0.50, the claim is inconclusive pending additional runs.

6.5.4 Implications for Framework Reliability

The multi-run evaluation serves two purposes. First, it provides robustness evidence: if the 95% CI for a dimension is narrow and consistently above a quality threshold, the framework produces reliable signals despite AI non-determinism. Second, it reveals which dimensions are most stable across runs. Structural Integrity may show high variance if the AI places files inconsistently; Type & Contract Safety may be more stable if Zod schema generation is driven by reliable TypeScript patterns. These patterns inform where framework enforcement mechanisms are most and least effective.

The single-run pilot presented in Section 6.7 was a feasibility-stage design selected for Bachelor’s thesis scope. The multi-run protocol documented here is the first priority for future empirical extension, as it directly addresses the non-determinism threat documented in the published literature.

6.6 Evaluation Metrics

Quantitative metrics derive from automated analysis of generated code. *Structural Integrity* uses Biome custom rules to check Next.js-specific file placement, naming, and Server/Client boundary conventions. *Type & Contract Safety* uses `tsc -noEmit` output to identify type violations and custom Biome rules for *any*-type detection and Zod coverage. *Coverage & Completeness* uses file enumeration against a reference checklist per feature type.

Qualitative metrics derive from expert review of generated code. Reviewers assess whether generated code follows Next.js best practices, whether naming conventions are consistent, and whether the overall implementation aligns with framework expectations. The expert review layer follows an inter-rater reliability (IRR) protocol: two independent raters score the same generated artifacts, with a target Cohen’s $\kappa \geq 0.7$. The automated scoring layer (Biome, `tsc -noEmit`) is deterministic and requires no IRR. The IRR protocol is documented here as a design-stage design; execution of the IRR pass is identified as future work pending availability of a second rater.

Productivity metrics follow the Meta DX framework (**beller2026dat**), measuring diff authoring time and developer-reported productivity. Task completion time measures the duration from task description to passing test suite. Code quality metrics include cyclomatic complexity, maintainability index, and code smell density.

6.7 Pilot Evaluation Protocol

This section describes the pilot evaluation conducted under the pre-registered adaptive- N protocol on the `isEnabled(flagKey, userId?)` utility in both Condition A (SmartKit with full convention encoding) and Condition B (convention-naïve scaffold). The final tally is 5 runs in total, distributed as $n = 2$ Cond A and $n = 3$ Cond B after the adaptive trigger fired on Cond B Type & Contract range (see §6.4 for the run-order schedule and the Adaptive N paragraph below for the adaptive decision). The pilot is documented as a worked-example, not as a hypothesis test. The per-arm sample size is below the $n = 5$ threshold the c2 §5.4.1 verdict rule requires for confirmed/refuted; the pilot demonstrates framework applicability and produces per-condition medians plus directional predictions (DP1–DP3), but cross-tool replication, larger- N validation at $n = 5$ per arm, and developer-participant studies remain future empirical work.

Task. The pilot applied the framework to a representative task: generating the `isEnabled(flagKey, userId?)` utility on SmartKit using Cursor AI. This utility resolves a feature flag with three override layers (per-user DB override, per-flag environment variable, default value) and caches the result in-memory for five minutes. The feature was chosen because it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), eliminating integration-pattern confounding identified in the C2 direction document. The task instruction mirrored a realistic developer request without over-specifying implementation details, allowing evaluation of the AI assistant’s ability to discover and apply existing conventions in the codebase.

Scoring. *Structural Integrity* was scored by running Biome with custom rules against the generated files and counting violations per category (file placement, naming, Server/Client boundary). *Type & Contract Safety* was scored by running `tsc -noEmit` on the generated code and counting type errors and *any-type* occurrences. *Coverage & Completeness* was scored by enumerating generated files against a reference checklist (utility function, Zod schema, colocated test, public API barrel) and computing a coverage ratio. Each dimension was reported on the 0–5 rubric described in Section 6.1 and Section 6.3.

Pilot Output. The pilot produced: (1) the task prompt (pre-registered and frozen before first run; SHA-256 recorded in the pre-registration §11 footer); (2) generated code snapshots for all 5 completed runs; (3) a scoring table per dimension per run; (4) per-condition medians, ranges, and Wilson 95% confidence intervals on the binary pass rate ($\geq 3/5$); (5) a DP-verdict document and the run-order schedule with adaptive- $N=3$ decision recorded. The full pilot artifacts (prompt, generated code, scoring scripts, pre-registration document, run log, adaptive decision, DP verdict) are released in `runs/` alongside this thesis.

Adaptive N. The pilot used the pre-registered adaptive protocol: 3 runs in Condition B were executed first, the inter-run range on each dimension was computed, and the Type & Contract Safety range (0–3 = range 3) exceeded the §3.4 threshold of 2, triggering PROCEED to Runs 4–5 in Condition A. The adaptive decision is documented in `runs/adaptive-decision.md` and was committed before Run 4 opened (ba694ab). The lock-time pre-registration and run-order schedule are preserved; only an additional run was triggered, consistent with the §3.4 protocol.

Per-run scores. Manual ratings per the rubric in `docs/pilot-pre-registration.md` §5:

Table 6.3: Pilot scoring output (actual data from completed runs, plus 3 pending Cond A runs per §A.11 amendment 2026-09-06). $N=3$ for Condition B (Runs 1–3); $N=2$ completed for Condition A (Runs 4–5); $N=3$ *pending* for Condition A (Runs 6–8) per adaptive N and the §A.5.4.1 verdict rule ($N = 5$ per condition).

Run	Cond	Structural	Type & Contract	Coverage	t_corr (min)	No
1	B	2	1	1	2	root lib/feature
2	B	2	3	1	2	root lib/flags.
3	B	2	0	1	2	root lib/feature-flag.ts
4	A	4	5	5	3	features/flags/lib +
5	A	4	5	4	3	features/flags/lib +
6	A	<i>pending</i> — to be executed per §A.11 amendment 2026-09-06; seed 1234567899				
7	A	<i>pending</i> — to be executed per §A.11 amendment 2026-09-06; seed 1234567900				
8	A	<i>pending</i> — to be executed per §A.11 amendment 2026-09-06; seed 1234567901				

Per-condition aggregate. Condition A medians [min, max]: Structural 4 [4, 4]; Type & Contract 5 [5, 5]; Coverage 4.5 [4, 5]. Condition B medians [min, max]: Structural 2 [2, 2]; Type & Contract 1 [0, 3]; Coverage 1 [1, 1]. The Condition B Type & Contract range (3) is the largest variance observed; the other two dimensions were stable within each condition.

DP verdicts (per docs/c2-direction §5.4.3 template). Three directional predictions (DP1: location; DP2: Zod enum; DP3: colocated test) are reported in three-statement form (k/n + Wilson 95% CI + Bonferroni-corrected comparative CI). The binary verdict rule under §5.4.1 requires $k/5$ runs in the predicted direction for *confirmed* or $0/5$ for *refuted*; values $1-4/5$ are *inconclusive*.

- **DP1 (Structural placement in features/flags/lib).** Cond A: 2/2 in predicted location. Cond B: 3/3 at root lib/ (Cond B's predicted location; equivalent to 0/3 in Cond A's predicted location). Wilson 95% CI: Cond A [0.34, 1.00]; Cond B [0.44, 1.00]. Comparative Bonferroni 99% CI: omitted per c2 §5.4.3 (reported only when both conditions reach adaptive- $N=5$; here $n = 2$ Cond A, $n = 3$ Cond B). **Verdict: inconclusive under §5.4.1 (sample too small per condition to apply the 5/5 or 0/5 binary rule).**
- **DP2 (Zod-derived FlagKey enum).** Cond A: 2/2 in Zod-derived FlagKey enum. Cond B: 3/3 in raw string type alias (Cond B's predicted location; equivalent to 0/3 Zod-derived). Wilson 95% CI: Cond A [0.34, 1.00]; Cond B [0.44, 1.00]. Comparative Bonferroni 99% CI: omitted per c2 §5.4.3 (reported only when both conditions reach adaptive- $N=5$; here $n = 2$ Cond A, $n = 3$ Cond B). **Verdict: inconclusive under §5.4.1 (sample too small per condition to apply the 5/5 or 0/5 binary rule).**
- **DP3 (Colocated test in src/tests/features/flags/).** Cond A: 2/2 in src/tests/features/flags/ (colocated). Cond B: 3/3 at root-level test, not colocated (Cond B's predicted location; equivalent to 0/3 colocated). Wilson 95% CI: Cond A [0.34, 1.00]; Cond B [0.44, 1.00]. Comparative Bonferroni 99% CI: omitted per c2 §5.4.3 (reported only when both conditions reach adaptive- $N=5$; here $n = 2$ Cond A, $n = 3$ Cond B). **Verdict: inconclusive under §5.4.1 (sample too small per condition to apply the 5/5 or 0/5 binary rule).**

Status Summary. The pilot demonstrates that the framework can measure AI-generated Next.js code on three dimensions and produce per-run scores. All five runs completed without exclusion events (E1–E5) and within the 60-minute time-to-correct budget per run. The framework property (c2 §5.3) is therefore empirically supported: the framework successfully measured all three dimensions in 5/5 runs.

The three directional predictions yielded *inconclusive* verdicts under the tightened rule (c2 §5.4.1, revised 2026-09-01) because the available per-condition sample ($n = 2$ Cond A; $n = 3$ Cond B) was below the $n = 5$ per arm threshold the rule requires for confirmed/refuted. Per-condition Wilson 95% CIs are reported per §5.4.3; the comparative Bonferroni 99% CI on the A – B difference is omitted under §5.4.3's gating rule (reported only when both conditions reach adaptive- $N=5$). At the point-estimate level, both conditions performed in their own predicted direction (Cond A 2/2, Cond B 3/3), which signals directional differentiation between the conditions but is not statistically powered at $n < 5$ per arm.

The pilot is a worked-example and is reported as such: the framework produces reproducible per-run scores on real AI-generated code under a pre-registered rubric and prompt,

and the locked pre-registration § 11 amendment log records every change with reason. The underpowered verdict does not retrofit a narrative; the inconclusive label is the honest outcome under §5.4.1 with the available data.

7. Conclusion and Future Work

7.1 Summary

This thesis targets the absence of an evaluation framework for AI coding assistant performance on Next.js boilerplate generation. The SmartKit framework supplies a structured architecture built around feature-based modularity, explicit Server/Client boundaries, and type-safe contracts. The evaluation framework provides a three-dimension assessment methodology for boilerplate quality that extends beyond existing benchmark methodology to measure convention adherence, type safety, and coverage.

The thesis proposes a Three-Dimensions Evaluation Framework for assessing AI-generated Next.js boilerplate quality across three dimensions: Structural Integrity, Type & Contract Safety, and Coverage & Completeness. The framework is intended for assessing convention adherence rather than algorithmic correctness, addressing a gap in existing benchmarks ([chen2021evaluating](#); [jimenez2024swebench](#); [rigorbench2026](#)). Two supporting artefacts accompany the framework proposal: the SmartKit implementation, a working Next.js App Router codebase illustrating the architectural approach; and the pilot evaluation protocol with $N=5$ runs per condition and a baseline comparison design, a documented procedure for applying the framework with replication artefacts (scripts, scoring rulebook, prompt templates, condition-bootstrap scripts). Validation of the framework—through executed multi-run experiments with statistical confidence intervals, cross-tool replication, and developer-participant studies—remains future empirical work. This thesis contributes the framework proposal, the published artefact, and the pilot protocol and baseline comparison design as a foundation for that empirical study.

7.2 Limitations and Threats to Validity

The proposed framework carries known design assumptions and identified threats to validity.

Construct Validity. The three evaluation dimensions are derived from design principles that address a gap RigorBench (2026) identifies in existing benchmarks: they measure algorithmic correctness and engineering process discipline but not convention adherence in generated boilerplate ([rigorbench2026](#)). The foundation comes from a structured literature search (Section 3), but the specific dimension selection and rubric cutoffs are design choices rather than empirically validated constructs. The rubric cutoffs (0–1: serious violations, 1–2: minor, 2–3: acceptable, 3–4: good, 4–5: excellent) are asserted mappings to developer outcomes and have not been calibrated against an inter-rater reliability study. Mitigation: the pilot protocol (Section 6.7) documents the scoring procedure for future empirical execution. Multi-run evaluation, IRR with $\kappa \geq 0.7$, and criterion-referenced validation are documented as follow-up work.

External Validity / Generalizability. The SmartKit implementation focuses exclusively on Next.js App Router. Architectural patterns specific to Next.js (Server Components, App Router conventions) may not transfer directly to other frameworks. The cross-framework transfer analysis (Section 7.4) provides a conceptual roadmap for Nuxt and Remix instantiation. Future work should evaluate generalizability across Nuxt and Remix by applying the

same three-dimension framework with framework-specific checklists; the cross-section is at present unverified.

Alternative Solutions. This thesis proposes a structural approach (project conventions + evaluation framework) but does not empirically compare against prompting strategies, enhanced documentation, or linter-based approaches. The rationale for the structural approach is discussed in Section 3, but no quantitative comparison exists. Mitigation: the Related Work section explicitly compares approaches and justifies the structural choice. Future empirical work should compare effectiveness across approaches.

Lab-to-Production Gap. The evaluation methodology uses scenario-based assessment rather than production deployment. Synthetic benchmark tasks may not capture the full complexity of real-world development workflows. Mitigation: the framework addresses this concern through trajectory-based evaluation (assessing code generation process, not just output), multiple runs to handle non-determinism, and planned developer participant studies following established methodology ([peng2023impact](#); [tariq2026enterprise](#)).

Scope limitations. The evaluation focuses exclusively on Next.js, and findings may not generalize to other frameworks. Longitudinal effects of AI-assisted development remain unexplored.

Snapshot limitation. The pilot measures one AI tool (Cursor AI) at one model snapshot, as recorded in the pre-registration §11 tooling block. Scores are not a stable measurement across AI tools or across model versions of the same tool; cross-tool and cross-version replication is identified as future work (Section 7.4). Readers should not extrapolate the DP verdicts in Section 6.7 beyond the single tool-snapshot pair that produced them.

Adaptive N caveat. The pre-registered “look after $n = 3$, decide whether to run additional runs” rule is a planned interim look, not a hypothesis test. No formal α -spending is applied because no hypothesis test is pre-registered; the decision at the interim look is to gather more descriptive data, not to test a directional hypothesis. The DP verdicts in Section 6.7 are descriptive under the binary rule (c2 §5.4.1): confirmed requires 5/5 in the predicted direction, refuted requires 0/5, and 1–4/5 are reported as inconclusive. The executed pilot yields inconclusive verdicts on all three DPs at $n = 2$ Cond A and $n = 3$ Cond B; this is the honest outcome under the rule, not an artefact of an underpowered significance test.

Inter-Rater Reliability (IRR) deferred. The pre-registered IRR protocol ([docs/pilot-pre-registration](#) §3.5; [latex/appendix-pre-registration.tex](#) §A.3.5) calls for a second rater to independently score 20% of runs (1 per condition) and a Cohen’s $\kappa \geq 0.7$ threshold. Execution of the IRR pass was deferred per amendment 2026-09-04 (recorded in §A.11) due to the absence of a second rater within the Bachelor’s thesis timeline. The manual cells in `runs/cond-{a,b}/run-NN/score.json` therefore reflect a single primary rater; the rubric-revision clause in §3.5 remains in force for any future IRR pass but was not exercised. The deferral is an acknowledged limitation: the per-dimension 0–5 cells in Table 6.3 are reproducible from the `score.json` files but have not been validated against a second rater.

Run-order confound. The locked schedule (B, B, B, A, A) runs all Cond B sessions first and all Cond A sessions last. This sequential run order means the pilot cannot rule out session-memory, time-of-day, or experimenter-fatigue confounds: an early Cond B session may have differed from a later Cond B session for reasons unrelated to the codebase being observed. A fully randomized design (e.g., a Latin square alternating conditions across the day) is identified as future work (§7.4); the adaptive-N=3 protocol that triggered PROCEED to Cond A was itself a function of the Cond B Type-range signal, so randomizing at the run level would conflict with the §3.4 verdict rule unless that rule is also revised.

7.3 Diffusion Pathway and Adoption Risks

A practical dimension of this work is how SmartKit might spread beyond a single author’s personal projects. Drawing on Rogers’ diffusion of innovations framework (**rogers2003diffusion**) as applied to developer tooling (**storey2016frameworkadoption**), the adoption profile of SmartKit depends on five attributes: relative advantage, compatibility, complexity, trialability, and observability.

Relative advantage for the early-majority segment is plausible but unquantified. The framework is intended to reduce boilerplate setup time and improve AI assistant adherence to Next.js conventions; both effects remain to be measured against no-framework and competing-framework baselines.

Compatibility with Next.js is high by construction (SmartKit is a Next.js boilerplate). **Complexity**, however, is substantial: developers adopting SmartKit must internalize feature-based folder conventions, Next.js Server/Client boundary directives (`'use client'` and `server-only` imports, which are Next.js defaults rather than SmartKit-specific syntax), shared Zod schema patterns, and the AGENTS.md instruction format. This overhead targets the innovator and early-adopter segments of the developer community; whether the framework can spread to the early majority depends on whether the maintenance overhead is amortized across projects.

Trialability is well-addressed: the SmartKit codebase is publicly released, and incremental adoption (taking only the feature-based layout and Server/Client boundary conventions without committing to the full stack) is documented. **Observability** is partially addressed: the proposed three-dimension rubric provides numeric scores per dimension, which can serve as an observability signal in published comparisons.

Adoption risks. Prior work on framework adoption identifies framework lock-in as a major adoption risk (**storey2016frameworkadoption**); SmartKit may intensify this risk by introducing a layer of project-specific conventions above plain Next.js. If a future Next.js major release changes its Server/Client boundary conventions (currently `'use client'` directives and `server-only` imports)—or if the conventions are folded into mainstream documentation—the SmartKit conventions may need to be migrated or retired. Teams that adopt SmartKit incrementally incur a partial lock-in: they may find rewriting their `features/` layout or unwiring shared Zod schemas across feature boundaries to be a non-trivial refactor. The thesis does not measure exit cost; future work should document a migration runbook.

Developer deskilling. A separate concern identified in adjacent work is whether structural boilerplate frameworks erode the developer expertise needed to debug, maintain, or evolve the generated code (**dakhel2023github**). Dakhel et al. find that AI assistance can harm novice developers who cannot identify suboptimal suggestions. SmartKit’s explicit, named conventions (“always put Zod schemas in `schemas/`”) are arguably more legible than AI-generated invisible structure, which can partially mitigate this risk: a developer who reads the AGENTS.md rules can verify that AI-generated code follows them, even when they would not have produced identical code from scratch. Future work should characterize this mitigation empirically by comparing developer debugging time on SmartKit-generated vs unstructured Next.js codebases.

7.4 Future Work

Cross-tool evaluation will replicate the pilot using alternative AI coding assistants (GitHub Copilot, Claude Code, Cody, Continue) to assess whether the framework’s dimensional scores are tool-specific or generalize across AI tools. The directional predictions (DP1–DP3) can be tested on each tool, producing a per-tool verdict table. Cross-framework evaluation will assess whether similar architectural patterns improve AI interaction in Nuxt (Vue) and Remix environments. The three-dimension framework is designed with portability in mind, though each dimension requires framework-specific instantiation. **Structural Integrity** maps directly: in Nuxt, the conventions are file-based routing via `/pages/`, `definePageMeta()` for metadata, `useAsyncData()` for server data boundaries, and `.client.vue` suffix for client-only components; in Remix, they are flat routes via `/routes/`, `loader/action` functions as server-side entry points, and `ErrorBoundary` components for error handling. These conventions are structurally analogous to Next.js App Router conventions but use different syntax and file organization. **Type & Contract Safety** transfers directly because it operates at the language level (TypeScript strict mode) and API level (Zod schemas), which are framework-agnostic. **Coverage & Completeness** requires per-framework checklist instantiation: Nuxt checklists include `middleware/` files and `composables/` for shared logic, while Remix checklists include `utils/` and `styles/` conventions. Developer-participant studies will examine how SmartKit performs in production environments with diverse development teams, measuring task completion time, code quality, and developer-reported productivity following the methodology of Peng et al. ([peng2023impact](#)) and Tariq et al. ([tariq2026enterprise](#)). Empirical comparison of alternative approaches (prompting strategies, enhanced documentation, linter-based enforcement) will quantify the relative effectiveness of the structural approach taken in this thesis.

Concrete next steps: (1) complete the §A.5.4.1 requirement by executing the 3 pending Cond A runs (Runs 6–8 in `runs/cond-a/run-0{6,7,8}/`) so the verdict rule can issue a confirmed/refuted outcome on each DP rather than the current inconclusive label; (2) run the cross-tool replication on at least two non-Cursor assistants; (3) instantiate the Nuxt and Remix checklists in draft form for review; (4) pilot a developer-participant session with three developers on a single SmartKit codebase and report findings before scaling further.

Sample-size determination. The executed pilot ($n=2$ Cond A; $n=3$ Cond B) is feasibility-stage, not validation-stage. A sample-size calculation for a two-proportion comparison on the per-condition pass rate ($\geq 3/5$ binary) at effect size $h = 0.5$ (Cohen’s medium effect), $\alpha = 0.05$, and power 0.8 gives $n \approx 30$ per condition (chi-squared approximation). At a stricter effect size $h = 0.3$ (small), the requirement rises to $n \approx 90$ per condition. The §A.5.4.1 verdict rule (5/5 confirmed, 0/5 refuted, 1–4/5 inconclusive) is a binary rule on a small- N design; it is appropriate for the current feasibility-stage pilot but is not a substitute for a frequentist sample-size calculation when moving to validation-stage replication. The 3 pending Cond A runs will bring that condition to $N = 5$ and allow the §A.5.4.1 rule to issue a verdict, but the $N = 5$ ceiling is the rule’s structural limit—a confirmed verdict at $N = 5$ should not be over-interpreted as a strong inferential claim.

Empirical validation of the orthogonality claim. The three evaluation dimensions (Structural Integrity, Type & Contract Safety, Coverage & Completeness) are claimed in §1.3 to be *designed to be* orthogonal in the sense that they measure distinct aspects of generated code. This orthogonality is asserted at the design stage and is consistent with how the rubric cells are defined, but it has not been empirically validated. A Pearson correlation matrix across the three dimension scores, computed on $n \geq 30$ per condition, would distinguish

genuine orthogonality (correlations ≈ 0) from subtle coupling (e.g., a Strong-type-safety run that also tends to be a Strong-coverage run because both depend on overall AI effort). Such empirical validation is left as future work; the current pilot ($N = 5$ per condition post-extension) is too small to detect anything but very large correlations.

Re-execution of the IRR pass. A second-rater IRR pass over 2 runs (1 per condition) with Cohen's $\kappa \geq 0.7$ is the pre-registered validation step (`docs/pilot-pre-registration.md` §3.5). The pass was deferred in amendment 2026-09-04 (recorded in §A.11). Future work should re-execute the IRR pass once a second rater is available; the rubric-revision clause in §3.5 remains in force but was not exercised.

A. Pilot Pre-Registration — SmartKit C2

A.1 Pilot Summary

Field	Value
Pilot type	Demonstration (N=5 per condition, 2 conditions)
Pilot question	Does providing architectural structure documentation improve AI coding assistant adherence to Next.js conventions and type safety?
Feature scope	Feature flag utility: <code>isEnabled(flagKey, userId?)</code>
Conditions	A (SmartKit), B (baseline)
Independent variable	Presence/absence of <code>AGENTS.md</code> , <code>.cursor/rules/</code> , and <code>src/features/</code> convention documentation
Dependent variables	Structural Integrity score, Type & Contract Safety score, Coverage & Completeness score
Pilot lock date	2026-09-02T05:23:00Z
Pre-registration	This document (SHA-256 recorded in §12)
Reporting	Aggregated scores per condition with 95% CI; no p-values or significance tests

Table A.1: Pilot summary

A.2 Pilot Feature

The pilot evaluates AI assistant performance on generating a **feature flag utility**: a function that checks whether a given feature flag is enabled for a user.

Scope: The utility must:

- Accept a flag key (string) and optional user ID
- Return a boolean indicating whether the flag is enabled
- Include type-safe schema validation (Zod)
- Include a colocated test file

File deliverables expected:

```
src/features/flags/lib/is-enabled.ts
src/features/flags/schemas/flag-check-input.ts
src/features/flags/lib/is-enabled.test.ts
(Condition A only: inside src/features/flags/)
```

Reasons for choosing this feature:

- Representative of common Next.js utility patterns (accepts input, validates, returns output)
- Touches multiple evaluation dimensions: file placement (Structural), Zod schema (Type Safety), test coverage (Coverage & Completeness)
- Small enough to complete in one AI assistant session (estimated 5–10 minutes)
- No external API dependencies (reduces confounding variables)
- Clear success criteria (function signature, schema, test presence)
- Demonstrates feature-based organization in Condition A vs flat structure in Condition B

A.3 Conditions

A.3.1 Condition A (SmartKit)

Condition A uses the SmartKit codebase, which includes:

- **AGENTS.md** at repository root: documents architectural conventions
- **.cursor/rules/** directory: cursor-specific instruction files
- **src/features/{name}/** structure: feature-based organization
- **Existing features:** `features/auth/` and `features/billing/` as examples
- **Zod schemas:** colocated in `features/{name}/schemas/`
- **Server Actions:** in `features/{name}/actions/`
- **server-only** imports: explicit server boundary markers

The AI assistant sees this structure before generating the feature flag utility.

A.3.2 Condition B (baseline)

Condition B uses a baseline Next.js codebase with:

- **No AGENTS.md** or `.cursor/rules/`
- **Flat structure:** `lib/` at repository root for utilities
- **No src/features/** directory
- **TypeScript strict mode** enabled (parity with Condition A)
- **Same dependencies:** Next.js, Drizzle, Zod, Biome (tooling parity)
- **.env.example** with placeholders for SePay/Resend/Better-Auth

The baseline represents a typical Next.js project without explicit architectural guidance.

A.3.3 Run Schedule

Run	Condition	Timestamp	Seed
1	A	2026-09-14T14:00:00Z	1234567890
2	B	2026-09-14T15:00:00Z	1234567891
3	A	2026-09-14T16:00:00Z	1234567892
4	B	2026-09-14T17:00:00Z	1234567893
5	A	2026-09-14T18:00:00Z	1234567894
6	B	2026-09-14T19:00:00Z	1234567895
7	A	2026-09-14T20:00:00Z	1234567896
8	B	2026-09-14T21:00:00Z	1234567897
9	A	2026-09-14T22:00:00Z	1234567898
10	B	2026-09-14T23:00:00Z	1234567899

Table A.2: Pilot run schedule (N=5 per condition, alternating order)

A.3.4 Adaptive N=3 Verdict Rule

If all three directional predictions (DP1, DP2, DP3) are supported after N=3 runs per condition, the pilot stops early. Otherwise, complete all N=5 runs.

Verdict logic:

For each DP (1, 2, 3):

If all 3 Condition A runs > all 3 Condition B runs on the relevant dimension:

DP is "supported"

Else:

DP is "not supported"

If all 3 DPs are "supported" after N=3:

Stop pilot, report results with N=3

Else:

Continue to N=5, report results with N=5

A.3.5 Inter-Rater Reliability

A second rater will independently score 20% of runs (2 runs, 1 per condition) using the same rubric. Cohen's κ will be computed. If $\kappa < 0.7$, the rubric will be revised and all runs re-scored.

A.4 Prompt Template

The exact prompt given to the AI assistant in all runs:

Add a feature flag check utility for this Next.js codebase. The utility should expose a single function 'isEnabled(flagKey, userId)' that returns a boolean.

Resolution order (highest priority first):

1. Per-user DB override, only if 'userId' is passed.
2. Per-flag environment variable 'FLAG_<KEY>=true|false'.
3. Default value (passed as the second argument).

Constraints:

- Cache the result in-memory for 5 minutes, keyed by flag and (if present) userId. Cache must be process-local.
- The function should be a server-side utility; do not export it to the browser.
- Define a 'FlagKey' type that the function accepts. Use a Zod schema to derive it.
- Include at least 3 unit tests covering: env override, default value, and cache hit.
- Use TypeScript strict mode.

You may add files anywhere in the repository that you think is appropriate. The only requirement is that the public API is reachable as 'isEnabled' and the type 'FlagKey' is exported.

Prompt delivery: The prompt is pasted into Cursor AI chat in a fresh session with no prior context. The assistant has access to the full codebase via file tree and search. The prompt is identical for both conditions; the only difference is the codebase state the assistant observes (Cond A is the SmartKit repo with FSD conventions and example features; Cond B is the create-next-app baseline). Any deviation (e.g., rephrasing, adding a follow-up clarification) is an exclusion event under criterion **E4** (§6).

A.5 Scoring Rubric

Each run is scored on three dimensions. Each dimension is on a 0–5 rubric. The score is the integer level the run falls into (no fractional scores). Ties are broken by the lower level. Aggregate score per run is 0–15 (sum of three dimensions). The primary analysis reports median + range per condition (consistent with §6.7 body and with SmartKit/scripts/aggregate-dp.ts).

A.5.1 Dimension 1: Structural Integrity (0–5)

Level	Label	Description (all must hold)
0	Missing	No <code>isEnabled</code> function produced.
1	Severe	Function exists but in <code>app/</code> , <code>root lib/</code> , or <code>root /</code> . No <code>features/</code> folder used.
2	Partial	Function in <code>features/</code> subtree but in a wrong sub-folder (e.g. <code>features/flags/index.ts</code> for the implementation, or <code>features/flags/components/</code>).
3	Acceptable	Function in <code>features/flags/lib/is-enabled.ts</code> (or colocated in another <code>features/{name}/lib/</code>) AND exports a named <code>isEnabled</code> function (default export also acceptable). File presence alone is insufficient; the export must exist.
4	Good	Level 3 + <code>server-only</code> import present in <code>is-enabled.ts</code> .
5	Excellent	Level 4 + <code>public API barrel features/flags/index.ts</code> re-exports <code>isEnabled</code> .

Table A.3: Structural Integrity scoring rubric (0–5)

A.5.2 Dimension 2: Type & Contract Safety (0–5)

Sub-score combination: the dimension score equals **min(Type Safety, Contract Safety)** on the 0–5 scale. The “weakest-link” aggregation is asserted at design stage; future empirical work should validate that it tracks developer experience better than weighted average or geometric mean.

Type Safety sub-score

Level	Label	Description
0	Failing	<code>tsc -noEmit</code> produces > 5 errors.
1	Poor	3–5 <code>tsc</code> errors, or ≥ 3 : <code>any / as</code> any occurrences.
2	Fair	1–2 <code>tsc</code> errors, or 1–2 any occurrences.
3	Good	0 <code>tsc</code> errors, 0 any in <code>isEnabled / FlagKey</code> API, but <code>// @ts-ignore</code> or <code>branded-type-as-string</code> present.
4	Very good	0 <code>tsc</code> errors, 0 any in <code>public API</code> , 0 <code>@ts-ignore</code> . <code>FlagKey</code> is a string literal union.
5	Excellent	Level 4 + <code>FlagKey</code> is a Zod-derived enum (<code>z.enum([...])</code> or <code>branded type</code> with <code>z.brand<...>()</code>).

Table A.4: Type Safety sub-score rubric (0–5)

Contract Safety sub-score

Level	Label	Description
0	None	No runtime validation: invalid <code>flagKey</code> reaches the cache layer.
1	Trivial	Validation exists but only as <code>if (typeof !== 'string') throw</code> .
2	Manual	Manual union-type guard at function entry.
3	Zod present	Zod schema present in <code>features/flags/types/flags.ts</code> and parsed at function entry.
4	Typed result	Level 3 + parse error surfaces a typed result (not <code>throw</code>).
5	Single source	Level 4 + schema is the single source of truth for <code>FlagKey</code> (no parallel string literal union).

Table A.5: Contract Safety sub-score rubric (0–5)

A.5.3 Dimension 3: Coverage & Completeness (0–5)

Level	Label	Description (all must hold)
0	No test	No <code>is-enabled.test.ts</code> file.
1	Minimal	Test file present but < 3 cases, or 3 cases but all <code>expect(true).toBe(true)</code> style.
2	Incomplete	≥ 3 test cases but no cache hit case.
3	Adequate	≥ 3 cases including env override + default value + cache hit. Cache implementation present (5-minute TTL, but the timing logic may be approximate).
4	Per-user covered	Level 3 + per-user override path is implemented (not stubbed) and has a test.
5	Excellent	Level 4 + no <code>console.log</code> left in production path + test file at centralized path <code>src/tests/features/flags/is-enabled.test.ts</code> (not in <code>__tests__</code>).

Table A.6: Coverage & Completeness scoring rubric (0–5)

A.5.4 Aggregate Score

Each run receives a score from 0 to 15 (sum of three dimensions; per-dimension max 5). The primary analysis reports median + range per condition with 95% confidence intervals on per-condition pass rates (Wilson 1927, per `SmartKit/scripts/lib/wilson.ts`).

A.6 Exclusion Criteria

ID	Exclusion Criterion
E1	AI assistant crashes, times out, or refuses to generate code
E2	Generated code contains syntax errors that prevent TypeScript compilation
E3	AI assistant generates files in programming languages other than TypeScript
E4	Run is interrupted by operator error (network failure, manual termination)
E5	AI assistant explicitly states it cannot complete the task due to missing dependencies or configuration

Table A.7: Exclusion criteria for pilot runs

Handling exclusions: If a run meets any exclusion criterion E1–E5, it is discarded and replaced with a new run using the next sequential seed value. The replacement run is logged in the Pilot Run Log (§10) with a note referencing the excluded run.

A.7 Lock and Amendment Procedure

This document was locked on **2026-09-02T05:23:00Z** before any pilot runs were executed. The lock establishes the following commitments:

- The prompt template (§4) will not be modified
- The scoring rubric (§5) will not be modified except for inter-rater reliability failure (§3.5)
- The exclusion criteria (§6) will not be modified
- The run schedule (§3.3) will not be modified except for exclusions (§6)
- The adaptive $N=3$ verdict rule (§3.4) will not be modified

Permitted amendments:

1. **Rubric clarification:** If Cohen’s $\kappa < 0.7$ after inter-rater reliability check, the rubric may be revised for clarity. The revision will be logged in §11 with the old and new SHA-256 hashes.
2. **Exclusion replacements:** If a run is excluded (§6), a replacement run is logged in §10 with the new seed value.
3. **Typographical corrections:** Minor formatting or spelling fixes that do not change the scientific meaning may be made without updating the lock hash.

Lock hash: The SHA-256 hash of this document at lock time is recorded in §11. Any substantive amendment (rubric revision, exclusion) updates the hash and is logged with justification.

A.8 Snapshot Limitation

This pre-registration documents the pilot design *as executed*, not a prospective commitment made before the idea existed. The lock date (2026-09-02T05:23:00Z) reflects when the design was finalized, after preliminary exploration but before formal data collection.

What this document does:

- Freezes the pilot protocol to prevent post-hoc changes
- Provides an auditable record of the scoring rubric and exclusion criteria
- Enables transparent reporting of what was planned vs. what was observed

What this document does not claim:

- That the research question or hypothesis was formed before any preliminary testing
- That no informal pilot runs or exploratory work preceded this lock
- That the design represents a traditional prospective pre-registration timeline

This pilot uses a demonstration design (N=5 per condition) appropriate for feasibility assessment, not hypothesis testing. The lock serves to document the protocol before formal execution, ensuring that rubric scoring and analysis decisions are not influenced by observed results.

A.9 References

- docs/pilot-pre-registration.md — Source markdown file (locked 2026-09-02T05:23:00Z)
- docs/c2-direction.md — Chapter 2 design document (pilot scope and directional predictions)
- docs/inter-rater.md — Inter-rater reliability protocol (Cohen’s κ computation)
- smartkit/ — SmartKit boilerplate codebase (Condition A)
- baseline/ — Baseline Next.js codebase (Condition B, not yet created at lock time)
- docs/plans/05-script-score-flag.md — Scoring automation script plan
- .cursor/rules/smartkit-conventions.mdc — SmartKit architectural conventions (Condition A only)
- Next.js 15 documentation — <https://nextjs.org/docs>
- Zod 4 documentation — <https://zod.dev>

A.10 Pilot Run Log

The pilot executes an adaptive schedule: start with three Cond B runs to establish a baseline Type-range signal, then proceed to Cond A if the Type-range threshold (§3.4) is met. The schedule recorded below is the **actually executed** sequence. All 5 completed runs used Prompt B verbatim (§4) on the `cursor-ai/auto` model snapshot (§8.1 amendment). Scores are primary-rater manual cells (0–5 per dimension; see §5).

#	Cond	Start (UTC+7)	Model	Str	Type	Cov	Tot	DP1	DP2	DP3	ttc (min)
1	B	2026-09-05T13:14	cursor-ai/auto	2	1	1	4	n/a	n/a	n/a	2
2	B	2026-09-05T13:18	cursor-ai/auto	2	3	1	6	n/a	n/a	n/a	2
3	B	2026-09-05T13:22	cursor-ai/auto	2	0	1	3	n/a	n/a	n/a	2
4	A	2026-09-05T13:26	cursor-ai/auto	4	5	5	14	incon.	incon.	incon.	3
5	A	2026-09-05T13:30	cursor-ai/auto	4	5	4	13	incon.	incon.	incon.	3
6	A	<i>pending</i>	cursor-ai/auto	<i>to be executed per §A.11 amendment 2026-09-06 (extend Cond A to n = 5 per adaptive PROCEED rule)</i>							
7	A	<i>pending</i>	cursor-ai/auto	<i>to be executed per §A.11 amendment 2026-09-06</i>							
8	A	<i>pending</i>	cursor-ai/auto	<i>to be executed per §A.11 amendment 2026-09-06</i>							

Table A.8: Pilot run log. Columns: Str = Structural Integrity, Type = Type & Contract Safety, Cov = Coverage & Completeness, Tot = sum of three dimensions (0–15), DP1/DP2/DP3 = directional prediction verdicts (§A.5.4.1: *5/5 confirmed, 0/5 refuted, 1–4/5 inconclusive*), ttc = time-to-correct (minutes from session open to committed snapshot). All runs reached `score-flag.ts` completion with no exclusion event (E1–E5). Wilson 95% CIs on per-condition pass rates are reported in §6.7 (body).

Notes:

- The original 10-run alternating schedule (A,B,A,B,A,B,A,B) was revised to the 5-run adaptive schedule (B,B,B,A,A) per `docs/c2-direction.md` §4 and locked in pre-registration amendment 2026-09-02 (§A.11).
- The Cond A run count was extended from 2 to 5 per amendment 2026-09-06 (§A.11) to satisfy the §A.5.4.1 verdict rule ($N = 5$ per condition). Cond B remains at $n = 3$ because the Cond B Type-range signal (Runs 1–3, range = 3) exceeded the adaptive PROCEED threshold; no further Cond B runs are needed for the directional prediction test.
- Type range across Cond B Runs 1–3 is 3 ($= 3 - 0$), exceeding the §3.4 threshold (2). Adaptive PROCEED was logged in `runs/adaptive-decision.md` (commit `ba694ab`) before Run 4 opened.
- All completed runs have an exclusion code of `null`. Model column is `cursor-ai/auto` per §8.1 (Auto-mode amendment); the prompt (§4) and schedule were preserved across all sessions.

A.11 Amendment Log

All amendments to the locked pre-registration are recorded here in chronological order. Each row references the section amended, the exact change made, the justification (including a citation to the source-of-truth document where the change is detailed), and the date. Amendments are appended; existing rows are not edited. The source-of-truth amendment log

lives in docs/pilot-pre-registration.md §11; the LaTeX table below is a thesis-side rendering for the committee.

Date	Section Amended	Justification
2026-09-02	Initial lock	Pre-registration locked before pilot execution.
2026-09-01	§A.5.4.1 verdict rule	Rule tightened from ‘ $\geq 4/5$ confirmed; $\leq 1/5$ refuted; 2–3/5 null’ to ‘5/5 confirmed; 0/5 refuted; 1–4/5 inconclusive’ per docs/c2-direction.md §5.4.1. Reason: original rule’s Wilson 95% CI for 4/5 ([0.30, 0.99]) overlaps the null range; tightened rule raises the confirmed bar and aligns the refuted bar with the strict-falsification standard.
2026-09-01	§A.3.6 starter-comparison scope (Table 3.1 body)	Retitled caption from “Comparison of SmartKit against established Next.js starter templates” (5 starters) to scope limited to SmartKit vs create-next-app baseline. Reason: original cell-level claims were unverified beyond the audit-anchor footnotes; §3.6 prose updated to mark the table as “qualitative positioning.”
2026-09-01	§5 scoring rubric (0–2 $\rightarrow$ 0–5)	Rubric widened from 3-level (0–2) to 6-level (0–5) per dimension to match the score-flag.ts implementation, which emits path_match $\in \{0, 1\}$ and feature_complete $\in [0, 1]$ combined into an integer 0–5 score. Total per-run max changed from 6 to 15. The widest-spread dimension (Type & Contract Safety) uses a weakest-link aggregation $\min(\text{type_sub_score}, \text{contract_sub_score})$ on the 0–5 scale.
2026-09-01	§4 prompt template	Prompt revised from “Create a feature flag utility for SmartKit. Requirements: ...” (Prompt A) to “Add a feature flag check utility for this Next.js codebase. ...” (Prompt B), with explicit resolution-order bullet list, server-side constraint, FlagKey Zod-derivation, 3-test requirement, and TypeScript strict mode. Reason: Prompt A leaked the word “SmartKit” into the Cond B baseline session, which would invalidate the convention-encoding comparison. New SHA-256: 18b84fd4dffff09d600548833c11eaded469e5a689b50ba1c13dc9e85375e8f2.
2026-09-02	§10 run schedule	Schedule revised from 10-run alternating (A,B,A,B,A,B,A,B,A,B) to 5-run adaptive (B,B,B,A,A) per docs/c2-direction.md §4. Reason: adaptive $N=3 \rightarrow 5$ design with an evidence-gated PROCEED decision based on the Type & Contract Safety range. A fully randomized design is noted as future work (§7.4).
2026-09-04	§A.3.5 IRR (Inter-Rater Reliability)	IRR execution deferred to future work. Reason: no second rater was available within the Bachelor’s thesis timeline. The rubric-revision clause (§A.3.5) remains in force for any future IRR pass; manual cells in runs/cond- $\{a,b\}$ /run-NN/score.json capture primary-rater scores and are reproducible. The deferral is acknowledged in §7.2 (Limitations).
2026-09-05	§12 JSON output schema	Schema updated to match the actual score-flag.ts v1.2 output: snake-case field names (structural_integrity, not structuralIntegrity); per-dimension score range 0–5 (not 0–2); added automated, automated_scores, manual.naming_quality, time_to_correct_minutes, edit_list, did_not_converge, exclusion_code fields. naming_quality is captured but not used in the DP verdict rule (reserved for future work, §7.4).
2026-09-06	§10 run schedule (post-execution extension)	Cond A extended from $n=2$ to $n=5$ (3 additional runs pending) to satisfy the §A.5.4.1 verdict rule ($N=5$ per condition). Cond B remains at $n=3$ because the Cond B Type-range signal (Runs 1–3, range = 3) exceeded the adaptive PROCEED threshold; no further Cond B runs are required for the directional-prediction test. New runs are appended to §10 with timestamps marked <i>pending</i> until executed.

Table A.9: Amendment log (locked + 7 amendments). Source of truth: docs/pilot-pre-registration.md §11.

SHA-256 hashes:

- **Prompt B template** (verbatim string in §4; matches docs/pilot-pre-registration.md §4): 18b84fd4dffff09d600548833c11eaded469e5a689b50ba1c13dc9e85375e8f2
- **Lock-time commit-attestation** SHA-256 (manually transcribed at lock 2026-09-02T05:23:00Z from the pre-reg-lock-20260902 tag message body): ca9eb4b39240f01f603f4cc5cadab336d3

- **Recomputed content SHA-256** of the locked commit(`git show 0ac4b35:docs/pilot-pre-regi`
| `sha256sum, pre-amendment`): `bd32d752c64785441eb69b9e3f91e6e99015b081e3648d20f25`
- **Post-amendment content SHA-256** of `docs/pilot-pre-registration.md` at HEAD
(after amendments 1–5 were applied to the source markdown): `639dde974d52b04c922cffd105db3a`

Note: the lock-time commit-attestation hash and the recomputed content hash differ because the former was manually transcribed at the moment of locking, before the §11 amendment log was filled in. The post-amendment content hash reflects the source markdown after all amendments 1–5 were applied (the LaTeX appendix is rendered from this state). The two locked hashes are preserved for cross-verification; the §11 amendment log in the source pre-registration is the source of truth for all subsequent content changes.

A.12 JSON Output Schema

Each pilot run produces a JSON file at `runs/cond-{a|b}/run-{NN}/score.json`. The schema matches the output of `SmartKit/scripts/score-flag.ts` (§A.5.4). Field names are snake_case; per-dimension score ranges are 0–5 (sum 0–15).

Field	Type	Description
<code>run_id</code>	string	Run number, zero-padded ("04").
<code>tooling_snapshot.tool</code>	string	Tooling used ("cursor-ai").
<code>tooling_snapshot.model</code>	string	Model identifier ("auto").
<code>tooling_snapshot.date</code>	ISO date	Run execution date.
<code>automated.structural_integrity</code>	object	Sub-scores: <code>is_enabled_file</code> , <code>has_is_enabled_export</code> , <code>uses_server_only_import</code> , <code>has_use_client_in_util</code> , <code>path_match</code> (0/1), <code>feature_complete</code> (0–1), <code>score</code> (0–5).
<code>automated.type_contract_safety</code>	object	Sub-scores: <code>tsc_error_count</code> , <code>any_count</code> , <code>ts_ignore_count</code> , <code>flag_key_is_zod_enum</code> , <code>flags_ts_found</code> , <code>type_sub_score</code> (0–5), <code>contract_sub_score</code> (0–5), <code>score</code> (0–5; min of sub-scores).
<code>automated.coverage_completeness</code>	object	Sub-scores: <code>is_enabled_ts_present</code> , <code>is_enabled_test_ts_present</code> , <code>flags_ts_schema_present</code> , <code>index_ts_barrel_present</code> , <code>five_minute_cache_implemented</code> , <code>test_case_count</code> , <code>console_log_count</code> , <code>score</code> (0–5).
<code>automated_scores</code>	object	Convenience: { <code>structural_integrity</code> , <code>type_contract_safety</code> , <code>coverage_completeness</code> } (the three top-level dimension scores).
<code>manual.structural_integrity</code>	int 0–5	Primary-rater score for Dimension 1 (§5.1).
<code>manual.type_contract_safety</code>	int 0–5	Primary-rater score for Dimension 2 (§5.2).
<code>manual.coverage_completeness</code>	int 0–5	Primary-rater score for Dimension 3 (§5.3).
<code>manual.naming_quality</code>	int 0–5	Primary-rater score for naming quality (captured but not used in DP rule; reserved for future work, §7.4).
<code>manual.notes</code>	string	Free-text primary-rater notes.
<code>total_score</code>	int 0–15	Sum of three manual dimension scores; <i>reported in §6.7 Table 6.3 only</i> . Not a stored field — computed at reporting time.
<code>time_to_correct_minutes</code>	int	Wall-clock minutes from session open to committed snapshot.
<code>edit_list</code>	string[]	Human-applied edits before scoring (empty if no edits needed).
<code>did_not_converge</code>	bool	<code>true</code> iff the AI session did not reach a passing <code>tsc -noEmit</code> + test-run state within the time budget.
<code>exclusion_code</code>	string null	One of "E1" – "E5" if excluded (§6), else <code>null</code> .

Table A.10: JSON output schema for pilot runs (matches `SmartKit/scripts/score-flag.ts v1.2` output). Snake-case field names; per-dimension scores are 0–5.

Field provenance:

- `automated.*` is computed by `SmartKit/scripts/score-flag.ts` from the run snapshot (file tree, `tsc -noEmit`, rg pattern matches).
- `manual.*` is recorded by the primary rater after inspecting the same snapshot; it is the value used in all per-run reporting (§6.7) and DP verdicts.
- `automated_scores` mirrors the three `automated.*.score` values for convenience.
- `time_to_correct_minutes`, `edit_list`, `did_not_converge`, and `exclusion_code` are recorded by the run-execution harness (`SmartKit/scripts/run-harness.sh`).

Document metadata:

- Lock date: 2026-09-02T05:23:00Z
- Source file: `docs/pilot-pre-registration.md` (source of truth for the amendment log; rendered above as Table A.9).
- Content hashes: see §A.11 (locked and post-amendment SHA-256 values).